

Sensors & Systems

Trilateration-based Sensors

Jan Bendtsen, Torben Knudsen | Electronic Systems, 8th Semester

Section 1 Trilateration-Based Sensors & Applications

1.1 What is Trilateration?

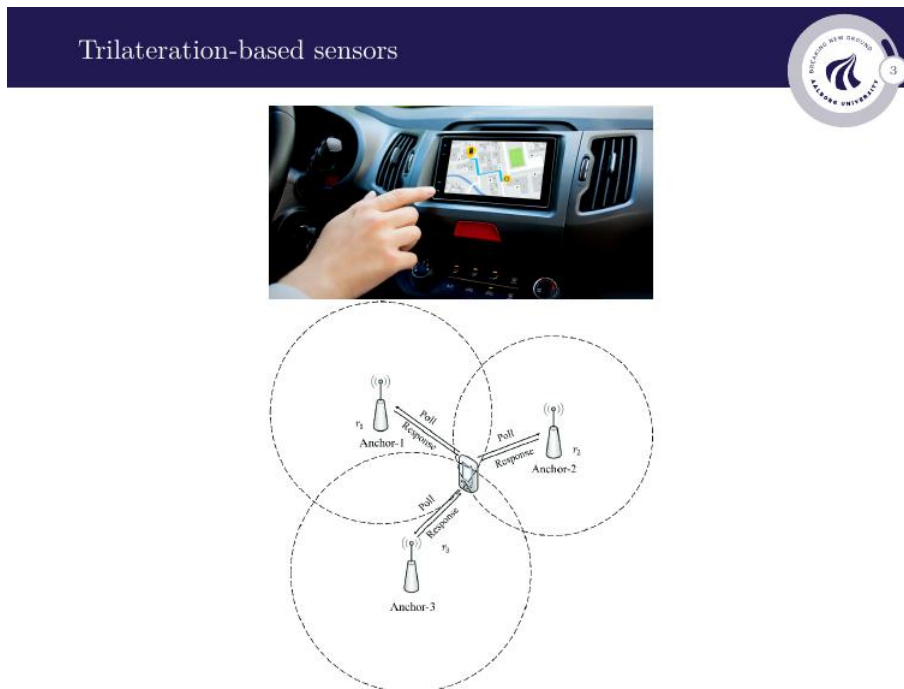


Figure 1: Trilateration principle. Three anchors at known positions each measure a range r_i to the unknown target (via a Poll/Response exchange). Each range defines a circle of possible target positions; the intersection of the three circles gives the unique target location.

Trilateration estimates the position of an unknown target using **range measurements** to several **anchors** — fixed points at known coordinates.

The core idea is simple:

- If you know you are exactly r_1 metres from Anchor 1, you could be anywhere on a *circle* (2D) or *sphere* (3D) of radius r_1 centred on Anchor 1.
- A second anchor narrows this to (at most) **two points** — the intersections of two circles.
- A **third anchor** resolves the ambiguity: only one point lies on all three circles simultaneously — that is the target's position.

How ranges are measured — Poll/Response

In most wireless trilateration systems, the anchor sends a *Poll* packet to the target and starts a timer. The target replies with a *Response* packet. The anchor measures the round-trip time Δt and computes the one-way range:

$$r = \frac{c \cdot \Delta t}{2}$$

where c is the propagation speed (speed of light for radio, speed of sound for acoustic systems). This is identical to the pulsed time-of-flight principle from Lecture 7, applied to wireless communication signals instead of lasers.

Trilateration vs. Triangulation

These two words are often confused:

- **Trilateration** uses *distances* (ranges) to known points.
- **Triangulation** uses *angles* to known points.

GPS, Bluetooth RSSI, and underwater acoustic positioning all use trilateration — they measure range, not angle.

Minimum number of anchors:

- **2D positioning:** 3 anchors (3 circles give a unique intersection).
- **3D positioning:** 4 anchors (4 spheres give a unique intersection).
- GPS requires 4 satellites even for 3D because the receiver clock error is an additional unknown — effectively a 4th dimension to solve for (covered in Section 4).

1.2 Bluetooth RSSI

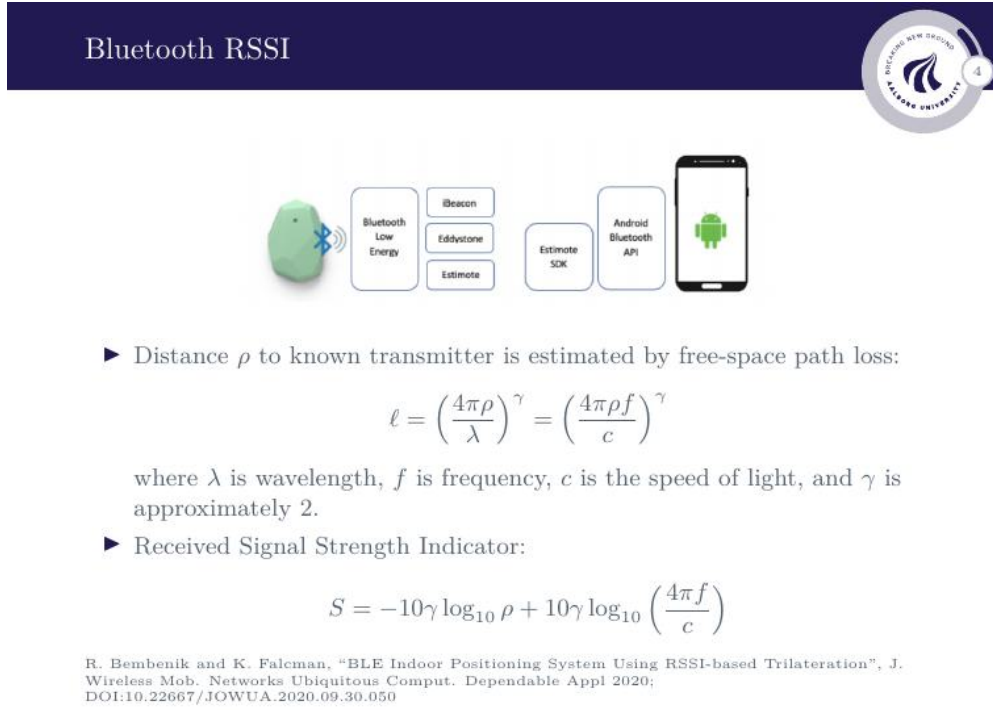


Figure 2: Bluetooth Low Energy (BLE) beacons act as anchors. The receiver (e.g. a smartphone) estimates its distance to each beacon by measuring the received signal strength. Multiple SDK options (iBeacon, Eddystone, Estimote) exist for the beacon protocol.

Bluetooth RSSI positioning uses **Bluetooth Low Energy (BLE)** beacons as anchors. Instead of measuring time of flight directly, it measures how much the signal has weakened over distance and uses that attenuation to estimate range.

Why use signal strength and not time-of-flight?

The obvious question: why not just send a signal, measure the round-trip time, and compute $r = c\Delta t/2$ — exactly as LiDAR and Games-on-Track do?

Radio travels too fast for standard BLE hardware. At $c = 3 \times 10^8$ m/s, the one-way travel time over 10 m is only:

$$t = \frac{10 \text{ m}}{3 \times 10^8 \text{ m/s}} = 33 \text{ ns}$$

To achieve 1 m range accuracy you would need to time this to ≈ 3 ns precision. That requires **dedicated nanosecond-precision timestamping circuits** built into the radio chip specifically for this purpose.

Standard BLE chips were designed for *data communication*, not precise timing. When a BLE chip receives a packet, there is a variable delay of **hundreds of microseconds** before software even knows the packet arrived (radio decoding, protocol processing, firmware overhead). The BLE specification sets a minimum reply turnaround time of

150 μs — with variation. At 150 μs of timing uncertainty alone:

$$\text{range error} = \frac{150 \times 10^{-6} \text{ s} \times 3 \times 10^8 \text{ m/s}}{2} = 22,500 \text{ m}$$

The processing delay completely swamps the useful travel time — time-of-flight is useless with off-the-shelf BLE hardware.

RSSI is used because it is free. Every BLE chip already measures received signal strength as part of normal radio operation (to assess link quality and decide whether to adjust transmission power). Converting that reading to a distance estimate costs nothing extra in hardware — just the log formula in software. It is not the most accurate method; it is simply the one that requires no additional hardware.

Time-of-flight systems (like Games-on-Track or modern UWB devices) use chips built from the ground up with dedicated hardware timestamp circuits that capture signal arrival to nanosecond precision. That is why they are more expensive and more accurate than BLE, but also why standard BLE cannot do the same.

Step 1 — Free-space path loss. When a radio signal travels a distance ρ through free space it loses power according to the **path loss** formula:

$$\ell = \left(\frac{4\pi\rho}{\lambda} \right)^\gamma = \left(\frac{4\pi\rho f}{c} \right)^\gamma$$

Symbol	Name	Physical meaning
ℓ	Path loss	Ratio of transmitted power to received power (dimensionless, > 1). Larger ℓ means more signal lost.
ρ	Range	Distance between transmitter and receiver (metres).
λ	Wavelength	$\lambda = c/f$; for BLE at 2.4 GHz, $\lambda \approx 12.5 \text{ cm}$.
f	Frequency	Carrier frequency (Hz); BLE uses $\approx 2.4 \text{ GHz}$.
c	Speed of light	$3 \times 10^8 \text{ m/s}$.
γ	Path loss exponent	≈ 2 in free space (theoretical). In buildings with walls and obstacles, γ is typically 2.5–4: signal decays faster than in free space.

What does path loss actually mean?

Path loss ℓ is the factor by which the received power is reduced relative to the transmitted power. For example, $\ell = 100$ means only 1 % of the transmitted power arrives at the receiver. The formula says: *doubling the distance multiplies the path loss by 2^γ* — at $\gamma = 2$, four times more power is lost every time you double the range. This is the same R^4 (or R^2 one-way) power decay discussed in the LiDAR scattering model of Lecture 7.

Step 2 — RSSI formula. The **Received Signal Strength Indicator** (RSSI) is the

received power expressed in **dBm** (decibels relative to 1 mW). Since transmitted power is constant, the RSSI decreases as range increases. Taking the path loss formula into the log domain:

$$S = -10\gamma \log_{10} \rho + 10\gamma \log_{10} \left(\frac{4\pi f}{c} \right)$$

Symbol	Name	Physical meaning
S	RSSI	Received signal strength in dBm. A less negative value means stronger signal (closer transmitter).
$-10\gamma \log_{10} \rho$	Range-dependent term	This is the part that grows (becomes more negative) as ρ increases. It is the “ruler” we use to estimate distance.
$10\gamma \log_{10}(4\pi f/c)$	Calibration constant	Depends only on frequency — fixed for a given beacon. In practice this is measured once at a known reference distance and stored as a calibration value.

Inverting RSSI to get range

Rearranging the RSSI formula to solve for ρ :

$$\rho = 10^{\frac{S_0 - S}{10\gamma}}$$

where $S_0 = 10\gamma \log_{10}(4\pi f/c)$ is the calibration constant (the RSSI you would measure at 1 m distance). This inversion is what converts a signal strength reading into a distance estimate.

What γ controls: In free space $\gamma = 2$, but in practice γ must be *calibrated* for each environment. A larger γ (e.g. 3 or 4 in a cluttered office) means the signal drops off faster, so any given RSSI value corresponds to a shorter range than it would in free space. Getting γ wrong leads directly to ranging errors and therefore positioning errors.



R. Bembek and K. Falman, "BLE Indoor Positioning System Using RSSI-based Trilateration", J. Wireless Mob. Networks Ubiquitous Comput. Dependable Appl 2020; DOI:10.22667/JOWUA.2020.09.30.050

Figure 3: Indoor BLE positioning result (floor plan, scale 1 cm : 250 cm). The green dot is the true position; the red dots are **repeated trilateration estimates from the same fixed position** — the receiver stood still and estimated its location many times. Each attempt gave a different answer because the RSSI reading fluctuates over time even when nothing moves.

What do the red dots actually represent?

Each red dot is the result of **one complete trilateration at one moment in time**. The process repeats many times from the same physical position:

1. Receive the signal from each of the three beacons.
2. Read the RSSI (total received power) from each.
3. Convert each RSSI to a range estimate using the formula.
4. Trilaterate — one position estimate. That is one red dot.

Why does each dot land in a different place? The signal from each beacon does not arrive via just one path. It bounces off walls, the floor, and furniture, so **multiple copies** arrive at the antenna simultaneously — the direct-path copy plus all the reflected copies. RSSI measures the *total* power of everything arriving at once; it cannot separate the direct copy from the reflections. This blended power fluctuates moment to moment (a door opens, someone walks past, air shifts) as the reflections shift in and out of phase with the direct signal. The fluctuating RSSI gives a fluctuating range estimate for each beacon, which gives a different trilateration result each time.

Summary: the red dots are *not* one signal arriving via multiple paths. They are *many separate measurements* over time, each landing somewhere slightly different because the combined direct+reflected power was slightly different at each moment.

The cloud of dots reveals the **noise floor of RSSI positioning**: roughly 1–3 m of random error even when perfectly stationary.

RSSI limitations indoors

The free-space path loss model assumes a clear line of sight between transmitter and receiver. Indoors this assumption breaks down:

- **Multipath**: the signal bounces off walls and arrives via multiple paths, causing constructive/destructive interference that varies even if you stand still.
- **Absorption**: walls, doors, and people absorb part of the signal, making γ effectively higher and location-dependent.
- **Result**: typical indoor BLE trilateration accuracy is 1–5 m — useful for room-level localisation but not for precise positioning.

1.3 Games-on-Track



<https://www.youtube.com/@GamesOnTrack/videos>

- ▶ Uses radio ‘chirps’ to determine the distance to known beacons
- ▶ Used to detect and track model trains.
- ▶ Used in several student projects at AAU in the past.
- ▶ Unfortunately the company went belly-up...

Figure 4: Games-on-Track: a commercial radio-based localisation system that used short radio “chirps” (brief bursts) to measure distances between a mobile tag and fixed beacons. It was used to detect and track model trains on a track layout.

Games-on-Track was a commercial wireless positioning system that used **radio chirps** — brief, wide-bandwidth radio pulses — to measure the time of flight between a mobile *tag* (attached to the object being tracked) and several fixed *beacons* (anchors).

What is a radio chirp?

Think of a police siren going “wooooo” — it sweeps from a low pitch up to a high pitch over a short time. A radio chirp does exactly the same thing with radio frequencies: the signal starts at a low frequency and sweeps up to a high one in a fixed, known pattern.

How the receiver uses it to measure range:

1. The receiver generates its own identical copy of the sweep pattern.
2. It slides this copy forward and backward in time and cross-correlates it with the incoming signal.
3. The correlation output spikes sharply at exactly the time shift where the two patterns line up — that shift is the travel time Δt .
4. Range: $r = c \cdot \Delta t$.

Why is this more precise than a single-frequency pulse? A single tone (one frequency) is easy to detect but hard to timestamp — you can tell SOMETHING is there, but not exactly *when* it arrived. The chirp has a unique, changing pattern: you can find the exact moment it lines up with your replica, giving a much sharper arrival timestamp. More frequencies swept = wider bandwidth = finer time resolution ($\Delta t \approx 1/B$) — same principle as short pulses in pulsed LiDAR (Lecture 7).

- **Measurement principle:** the beacon transmits a chirp (a “poll”). The tag records the precise arrival time and the time difference gives the range via $r = c\Delta t$ (one-way) or $r = c\Delta t/2$ (round-trip, if the tag also replies). In either case, the receiver locks onto the **first-arriving** copy of the chirp.
- **Application:** primarily used to detect and track model trains in hobby layouts, and was used in several AAU student projects.
- **Status:** the company went bankrupt and the system is no longer commercially available.

Why “first arrival” beats RSSI indoors

When a radio signal travels indoors it splits into many copies: one travels the direct path, others bounce off walls, floors, and furniture and arrive slightly later.

RSSI receives *all* copies simultaneously — direct path and reflections add together in power. The total power fluctuates as the reflections shift in and out of phase, so the RSSI reading is noisy and environment-dependent.

Chirp / TOF locks onto the *first-arriving* copy only. The first copy always travelled the shortest path — the straight line between transmitter and receiver. Reflections arrive later (longer path) and are simply ignored. The first-arrival time therefore always gives the true direct distance, unaffected by how many reflections exist. This is why time-of-flight is far more accurate indoors than RSSI, at the cost of needing more precise timing hardware.

1.4 Underwater Sensor Networks

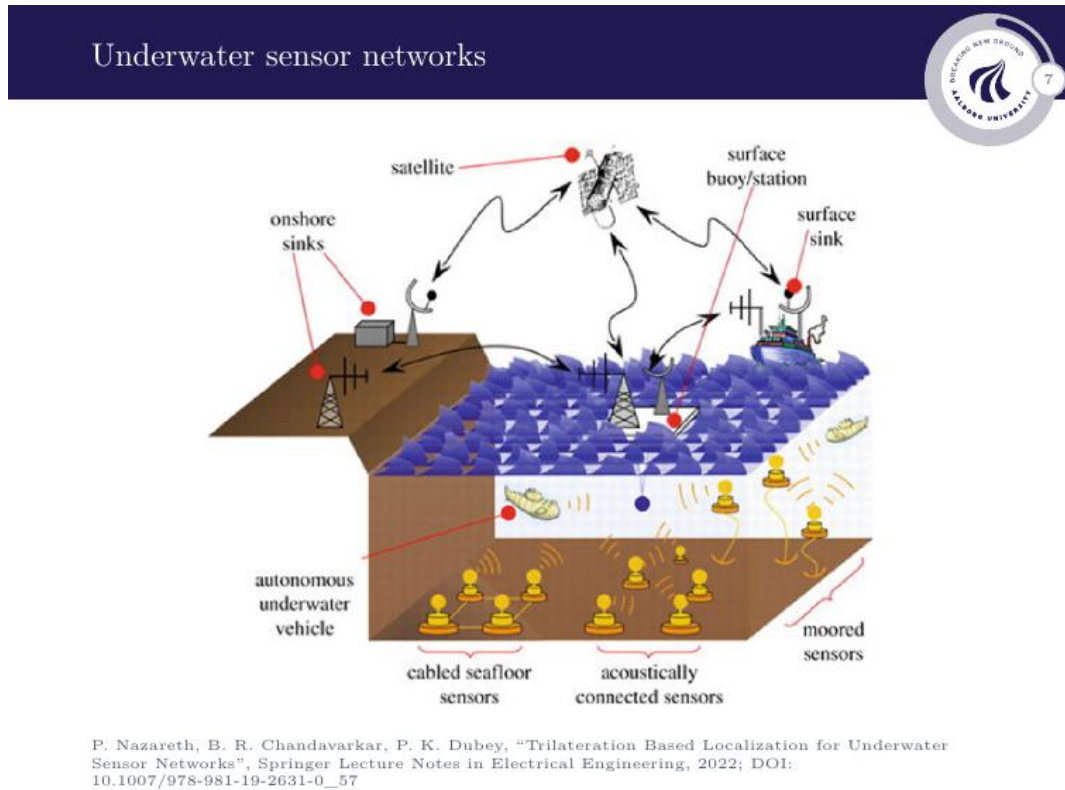


Figure 5: Underwater sensor network architecture. Moored seafloor sensors and autonomous underwater vehicles (AUVs) communicate acoustically (sound waves). Surface buoys relay data to satellites and onshore sinks. Trilateration with acoustic signals gives the AUV its position relative to the known anchor positions.

Trilateration is also used in **underwater environments**, but with a critical difference: radio waves do not propagate well in water (they are absorbed within a few metres). Instead, **acoustic signals** (sound waves) are used.

- **Sound speed in water:** approximately 1500 m/s (compared to 3×10^8 m/s for radio). The same Poll/Response range equation $r = c\Delta t/2$ applies, but with $c \approx 1500$ m/s.
- **Anchors:** moored seafloor sensors or surface buoys at known positions act as anchors.
- **Mobile nodes:** autonomous underwater vehicles (AUVs) use acoustic trilateration to determine their own position.
- **Surface relay:** once the AUV has computed its position from the acoustic trilateration, a surface buoy receives that result acoustically and re-transmits it upward via **radio** to a satellite or onshore station. Radio is *not* used for the ranging — only for relaying the already-computed position over long distances at the speed of light.

Two technologies, two roles — neither can replace the other

		Acoustic (sound)	Radio
Role		Trilateration — measures range between underwater nodes	Communication — relays the computed position to shore
Why this medium		Radio is absorbed within cm of salt water; sound travels hundreds of metres	Sound in air is slow and short-range; radio travels at c over thousands of km
Speed		≈ 1500 m/s	3×10^8 m/s
Domain		Underwater only	Surface / atmosphere / space

The surface buoy is the **bridge**: it listens acoustically below the water and transmits by radio above it. The two systems never overlap — radio never goes underwater, acoustic never carries the data to shore.

Why sound, not radio, underwater?

Radio waves are electromagnetic and are rapidly absorbed by the conductive salt water (skin depth is only a few cm at typical RF frequencies). Sound is a mechanical wave and propagates through water with much less attenuation over distances of hundreds or even thousands of metres. The trade-off is a much lower propagation speed (1500 m/s vs. 3×10^8 m/s), which limits the update rate of range measurements and requires careful compensation for sound speed variations with temperature and salinity.

Why each system measures range the way it does

All three systems do trilateration, but each uses a different method to measure range — and each choice is forced by the physics and hardware of that environment.

Bluetooth RSSI uses signal strength because it has no choice. Radio travels at the speed of light, so a 10 m indoor distance takes only 33 ns to cross. A standard BLE chip has processing delays of around 150 μ s — thousands of times longer than the travel time itself. Trying to time the signal would give over 22 km of range error from the chip delay alone, which is useless. But every BLE chip already measures received signal strength as part of normal operation, so RSSI costs nothing extra — just plug the reading into the log formula and you have a range estimate. It is not the most accurate method; it is the one that requires no additional hardware.

Games-on-Track uses time-of-flight because it can. The chirp hardware contains dedicated nanosecond-precision timestamping circuits built specifically to capture the exact arrival time of the first-arriving signal copy. This makes TOF both possible and accurate. It could in principle use RSSI instead, but why settle for a noisy amplitude reading when precise timing is available? The first-arriving chirp copy always travelled the direct path, so the range is clean and unaffected by reflections.

Underwater acoustic uses sound because radio is not an option underwater. Salt

water absorbs radio waves within a few centimetres, so there is no signal left to measure after even a short distance. Sound, on the other hand, travels hundreds of metres through water with manageable attenuation. The same TOF principle applies — measure when the sound pulse arrives, multiply by the speed of sound — just with $c \approx 1500 \text{ m/s}$ instead of $3 \times 10^8 \text{ m/s}$. Radio is then used only at the surface to relay the already-computed position up to a satellite or shore station, where it can actually propagate.

Section 1 — Key Takeaways

- **Trilateration** uses ranges to ≥ 3 known anchors to find an unknown position; each range constrains the target to a circle (2D) or sphere (3D).
- **Bluetooth RSSI**: range from signal strength via $S = -10\gamma \log_{10} \rho + \text{const.}$. Accurate outdoors ($\gamma \approx 2$), but noisy indoors due to multipath and absorption (typical error 1–5 m).
- **Games-on-Track**: time-of-flight radio chirps — more robust than RSSI because it uses the first-arrival time rather than signal strength.
- **Underwater networks**: radio replaced by acoustic signals ($c \approx 1500 \text{ m/s}$); same trilateration principle, but with slower propagation and speed variations with temperature and salinity.

Section 2 Earth Geometry & Coordinate Systems

Before GPS can tell you where you are, it needs a precise mathematical description of where everything else is — the Earth's shape, the satellites' positions, and how to express positions in a consistent coordinate system. This section builds that foundation.

2.1 Why GPS Needs 4 Satellites

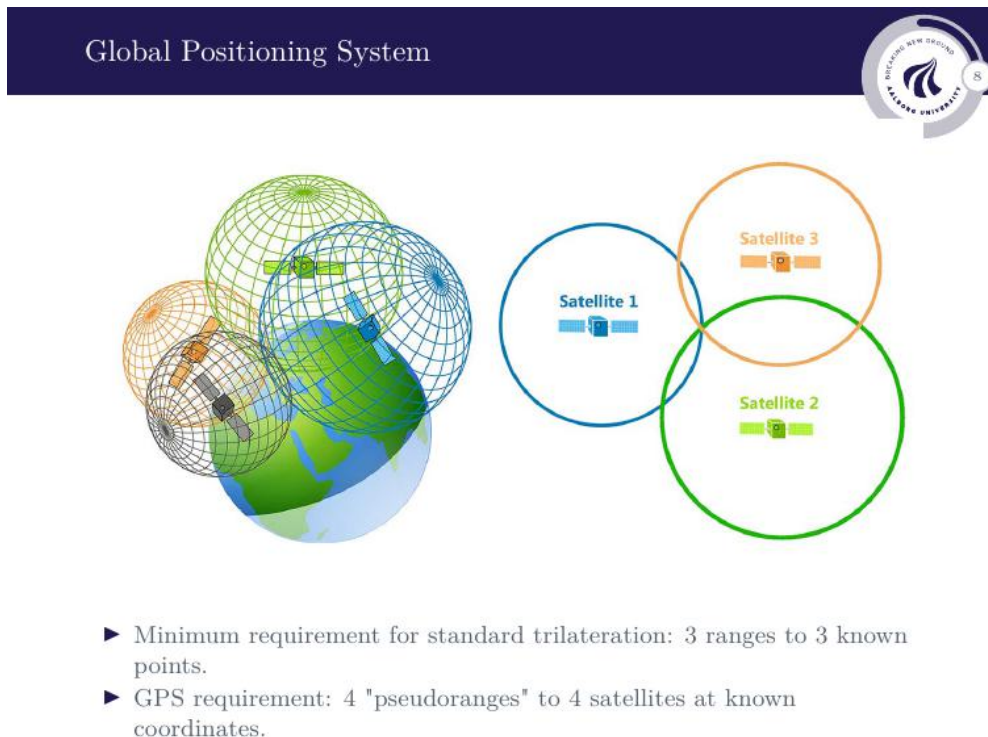


Figure 6: GPS trilateration in 3D. Each satellite defines a sphere of possible receiver positions. Three spheres intersect at (at most) two points; a fourth satellite resolves the ambiguity — and also solves for the unknown receiver clock error.

Standard trilateration in 3D needs **3 ranges to 3 known points** to give a unique position — the intersection of three spheres. GPS needs **4 ranges** because it has a fourth unknown: the **receiver clock error**.

Satellite clock vs. receiver clock — what are they?

The satellite is the GPS spacecraft orbiting at $\approx 26,000$ km altitude. Each satellite carries **atomic clocks** (rubidium and caesium) that are extraordinarily precise — they drift by less than a microsecond per day. The satellite uses its clock to *stamp the exact time* onto every signal it broadcasts.

The receiver is the GPS device on the ground — your phone, a car navigation unit, or a handheld GPS. It contains a cheap **quartz crystal oscillator**, the same kind used in a wristwatch. Quartz clocks drift by tens of microseconds per second and are

nowhere near as precise as the satellite’s atomic clock.

The receiver measures the travel time of each satellite’s signal by comparing the timestamp on the arriving signal with its own clock. Because its own clock is running slightly fast or slow relative to the satellite clocks, the measured travel time is always off by the same unknown amount τ — the **receiver clock error**. This makes every range measurement longer or shorter than the true value by $c\tau$.

The satellite clock also has a small error, but that is known — the satellite broadcasts a correction polynomial in its navigation message (covered in Section 3), so the receiver can remove it. The receiver clock error τ is *not* known and must be solved for alongside the position.

The measured distance is therefore called a **pseudorange** — it looks like a range but is corrupted by the unknown clock error. With 4 satellites there are 4 pseudorange equations and 4 unknowns: receiver position (x, y, z) and clock error τ .

Why “pseudo” range?

If the receiver clock were perfectly synchronised with the satellite clocks, the measured travel time would give the true geometric distance. Because the receiver clock runs slightly fast or slow, the measured travel time is always off by the same amount τ , making every range measurement too long or too short by the same $c\tau$. The measurement is called a pseudorange because it is *almost* a range — just shifted by an unknown constant. The fourth satellite equation lets the solver find and cancel out that shift, recovering both the true position and the clock error simultaneously.

2.2 The ECEF Coordinate System

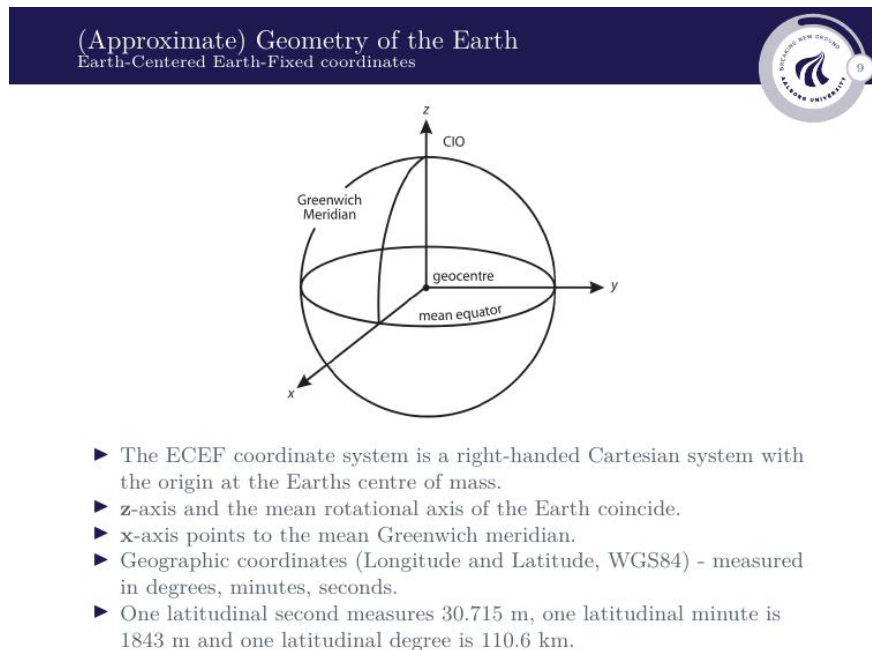


Figure 7: The Earth-Centred Earth-Fixed (ECEF) coordinate system. Origin at Earth's centre of mass; z -axis along the mean rotational axis (toward CIO); x -axis toward the Greenwich meridian. The system rotates with the Earth, so a fixed point on the surface always has the same (x, y, z) coordinates.

All GPS calculations happen in the **Earth-Centred Earth-Fixed (ECEF)** coordinate system — a right-handed Cartesian system defined as follows:

- **Origin:** Earth's centre of mass.
- **z -axis:** points along Earth's mean rotational axis toward the Conventional International Origin (CIO), i.e. approximately toward geographic north pole.
- **x -axis:** points toward the intersection of the equatorial plane and the mean Greenwich meridian (0° longitude).
- **y -axis:** completes the right-handed system (90° east of x in the equatorial plane).

What “Earth-Fixed” means — and why satellites are a problem

The ECEF frame *rotates with the Earth*. A building in Aalborg has the same (x, y, z) ECEF coordinates today and tomorrow — the frame spins with the planet so fixed points on the surface stay fixed in the coordinate system. This is convenient for the receiver on the ground.

Satellites, however, orbit in **fixed inertial space** — their orbital plane does not rotate with the Earth at all. This means a satellite's ECEF coordinates are *continuously changing* even if its orbit is perfectly stable, because ECEF is spinning underneath it at one full rotation per day.

How the receiver handles this: The receiver computes the satellite's position in inertial space using the Keplerian orbital parameters (from the Navigation Message), then applies an extra rotation by the **Earth's current rotation angle** at the exact time the signal was transmitted. Earth rotates at $\omega_E \approx 7.27 \times 10^{-5} \text{ rad/s}$, so even a 20 ms travel time causes the Earth to rotate by a small but non-negligible amount that must be corrected.

How the orbital parameters stay accurate: A Keplerian ellipse is only an approximation — the Moon, Sun, solar radiation, and atmospheric drag all slowly nudge the satellite off its ideal orbit. The GPS **Control Segment** (a network of ground stations at precisely known positions worldwide) continuously tracks all satellites, measures the small deviations from the predicted orbit, and runs estimation algorithms (essentially a Kalman filter) to compute corrected orbital parameters. These corrections are uplinked to each satellite several times per day. The satellite then broadcasts the updated parameters in its Navigation Message, so the receiver always has fresh, accurate data to compute satellite positions from.

Geographic coordinates. In everyday use, positions are given as longitude λ and latitude ϕ . These are angles, and angles can be written in two ways:

- **Decimal degrees** — e.g. 57.046° N
- **Degrees, Minutes, Seconds (DMS)** — e.g. $57^\circ 02' 46'' \text{ N}$

DMS works exactly like hours on a clock: one degree is split into 60 *minutes* ($'$), and one minute is split into 60 *seconds* ($''$). These are **angular units** — nothing to do with time — just a way of subdividing a degree into smaller pieces (just as an hour is subdivided into minutes and seconds).

What does a degree/minute/second of latitude mean in metres?

One degree of latitude corresponds to moving $\approx 110.6 \text{ km}$ along the surface of the Earth. Because there are 60 minutes in a degree and 60 seconds in a minute:

- 1 degree = 110.6 km
- 1 minute (1/60 of a degree) = $110,600 \text{ m}/60 \approx 1843 \text{ m} \approx 1.85 \text{ km}$
- 1 second (1/3600 of a degree) = $110,600 \text{ m}/3600 \approx 30.7 \text{ m}$

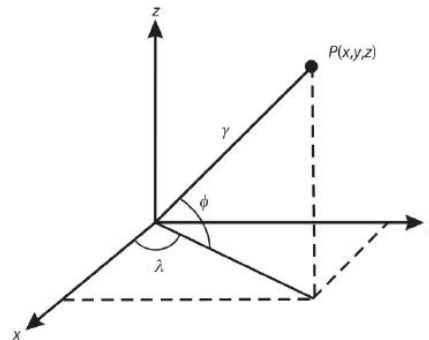
A practical example: Aalborg is at approximately $57^\circ 02' 46'' \text{ N}$, $9^\circ 54' 57'' \text{ E}$. Moving 1 second further north from there moves you about 30 m along the ground — roughly the length of a bus. Moving 1 minute north moves you about 1.85 km.

This is also where the **nautical mile** comes from: 1 nautical mile is defined as exactly 1 arcminute of latitude $\approx 1852 \text{ m}$, which is why it is still used in aviation and marine navigation.

Note that these distances apply *along lines of latitude* (north–south). Lines of longitude (east–west) converge toward the poles, so 1 degree of longitude in the east–west direction is shorter the further from the equator you are.

Converting between geographic and ECEF.

(Approximate) Geometry of the Earth Spherical coordinates



► Point P in ECEF (global) coordinates:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \gamma \begin{bmatrix} \cos \phi \cos \lambda \\ \cos \phi \sin \lambda \\ \sin \phi \end{bmatrix} \quad \text{and} \quad \begin{aligned} \gamma &= \sqrt{x^2 + y^2 + z^2} \\ \tan \lambda &= y/x \\ \tan \phi &= z/\sqrt{x^2 + y^2} \end{aligned}$$

where λ is longitude, ϕ is latitude and γ is often taken as R_E , the Earth's mean radius.

Figure 8: Spherical coordinates. A point P at distance γ from the Earth's centre, longitude λ , and latitude ϕ maps to ECEF Cartesian coordinates via the spherical conversion formula.

A point P at distance γ from the centre, longitude λ , and latitude ϕ has ECEF coordinates:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \gamma \begin{bmatrix} \cos \phi \cos \lambda \\ \cos \phi \sin \lambda \\ \sin \phi \end{bmatrix}$$

Symbol	Name	Physical meaning
λ	Longitude	Angle east/west of Greenwich meridian. Ranges from -180° to $+180^\circ$.
ϕ	Latitude	Angle north/south of the equator. Ranges from -90° (south pole) to $+90^\circ$ (north pole).
γ	Radial distance	Distance from Earth's centre. For points on the surface, $\gamma \approx R_E = 6,371 \text{ km}$ (Earth's mean radius).

The inverse — recovering λ , ϕ , γ from Cartesian (x, y, z) — uses:

$$\gamma = \sqrt{x^2 + y^2 + z^2}, \quad \tan \lambda = \frac{y}{x}, \quad \tan \phi = \frac{z}{\sqrt{x^2 + y^2}}$$

How latitude and longitude relate to the ECEF axes

It helps to think of the three ECEF axes as the three “anchor points” of the coordinate system:

z -axis = the north pole direction.

- Latitude ϕ is the angle you tilt *up from the equatorial plane toward the north pole*.
- $\phi = 0^\circ$ (equator) means you are entirely in the x - y plane, so $z = 0$.
- $\phi = +90^\circ$ (north pole) means you are sitting exactly on the z -axis, so $x = y = 0$ and $z = \gamma$.
- $\phi = -90^\circ$ (south pole) means $z = -\gamma$. Latitude is purely about how far you are from the equatorial plane.

x -axis = the Greenwich meridian at the equator.

- Longitude λ is the angle you rotate *east or west around the z -axis*, starting from the Greenwich meridian ($\lambda = 0^\circ$).
- The x -axis points to the exact spot where the equator and the Greenwich meridian cross — that is latitude 0° , longitude 0° , and it sits on the x -axis.
- Rotate east by 90° ($\lambda = 90^\circ$ E) and you reach the y -axis. Rotate to $\lambda = 180^\circ$ (the International Date Line) and you are on the negative x -axis.

y -axis completes the right-hand system and points to 0° latitude, 90° E longitude — in the Indian Ocean.

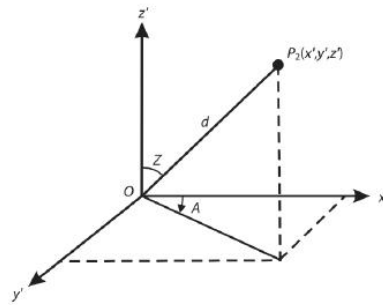
A quick way to check any point:

- North pole ($\phi = 90^\circ$, any λ): $[x, y, z] = [0, 0, \gamma]$ — purely on the z -axis.
- Equator + Greenwich ($\phi = 0^\circ$, $\lambda = 0^\circ$): $[x, y, z] = [\gamma, 0, 0]$ — purely on the x -axis.
- Equator + 90° E ($\phi = 0^\circ$, $\lambda = 90^\circ$): $[x, y, z] = [0, \gamma, 0]$ — purely on the y -axis.

2.3 Local Coordinate Systems

(Approximate) Geometry of the Earth

Local coordinates



- Let (x', y', z') be a local coordinate system. The x', y' plane¹ is normally referred to as the *horizontal plane*.
- A and Z are known as *Azimuth* and *Zenith* (angles), respectively.
- For point P_2 one has:

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = d \begin{bmatrix} \cos A \sin Z \\ \sin A \sin Z \\ \cos Z \end{bmatrix} \quad \text{and} \quad \begin{aligned} d &= \sqrt{x'^2 + y'^2 + z'^2} \\ \tan A &= y'/x' \\ \cos Z &= z'/d \end{aligned}$$

¹Obs: x' and y' should be swapped in the figure above.

Figure 9: Local coordinate system (x', y', z') centred at a reference point O on the Earth's surface. The $x'-y'$ plane is the local horizontal plane; z' points straight up (zenith). A is the azimuth (horizontal direction); Z is the zenith angle (angle from vertical); d is the slant distance to point P_2 .

ECEF is great for the maths but useless for a human standing on the ground. Nobody thinks “the satellite is at $x = 15,000$ km from Earth's centre.” You think: “the satellite is high up, roughly to the northeast.” A **local coordinate system** puts the origin at *your position* on the surface and defines axes that make sense from where you are standing:

- One axis pointing **N**orth along the ground.
- One axis pointing **E**ast along the ground.
- One axis pointing straight **U**p into the sky.

Think of it like this: when you hold your phone flat and open a compass app, the phone's screen is the local horizontal plane — North and East are two directions across the screen, and Up is perpendicular to the screen pointing at the ceiling. That is essentially a local coordinate system centred on where you are standing.

Why not just use ECEF for everything?

ECEF has its origin at Earth's centre thousands of kilometres below your feet, and its axes point toward the north pole and Greenwich. If you compute the GPS position error in ECEF and get a vector $[3 \text{ m}, -1 \text{ m}, 2 \text{ m}]$, it is very hard to tell whether that is mostly a horizontal error (you are misplaced sideways on the map) or a vertical error (the GPS thinks you are flying two metres above the ground). In a local North-East-Up frame the same error becomes $[\Delta\text{North}, \Delta\text{East}, \Delta\text{Up}]$, which immediately tells

you both the horizontal spread and the vertical offset — far more useful in practice. Converting ECEF errors to local coordinates is exactly what the rotation matrix $R_{\phi\lambda}$ in Section 4 does.

In the local frame, point P_2 at slant distance d , azimuth A , and zenith angle Z has local Cartesian coordinates:

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = d \begin{bmatrix} \cos A \sin Z \\ \sin A \sin Z \\ \cos Z \end{bmatrix}$$

Symbol	Name	Physical meaning
d	Slant distance	Straight-line distance from the local origin to the point. $d = \sqrt{x'^2 + y'^2 + z'^2}$.
A	Azimuth	Horizontal compass direction to the point, measured from the x' -axis (usually North). $\tan A = y'/x'$.
Z	Zenith angle	Angle measured from straight up (z' -axis) down toward the horizon. $Z = 0^\circ$ means directly overhead; $Z = 90^\circ$ means on the horizon. $\cos Z = z'/d$.

Azimuth and zenith in plain terms

Imagine standing outside and looking at a GPS satellite in the sky. **Azimuth** tells you which compass direction to face — “turn to face 45° north-east.” **Zenith angle** tells you how far to tilt your head up — $Z = 0^\circ$ means look straight up at the zenith; $Z = 45^\circ$ means tilt 45° down from vertical; $Z = 90^\circ$ means look at the horizon. Together they point your gaze exactly at the satellite; d tells you how far away it is.

Common local coordinate conventions.

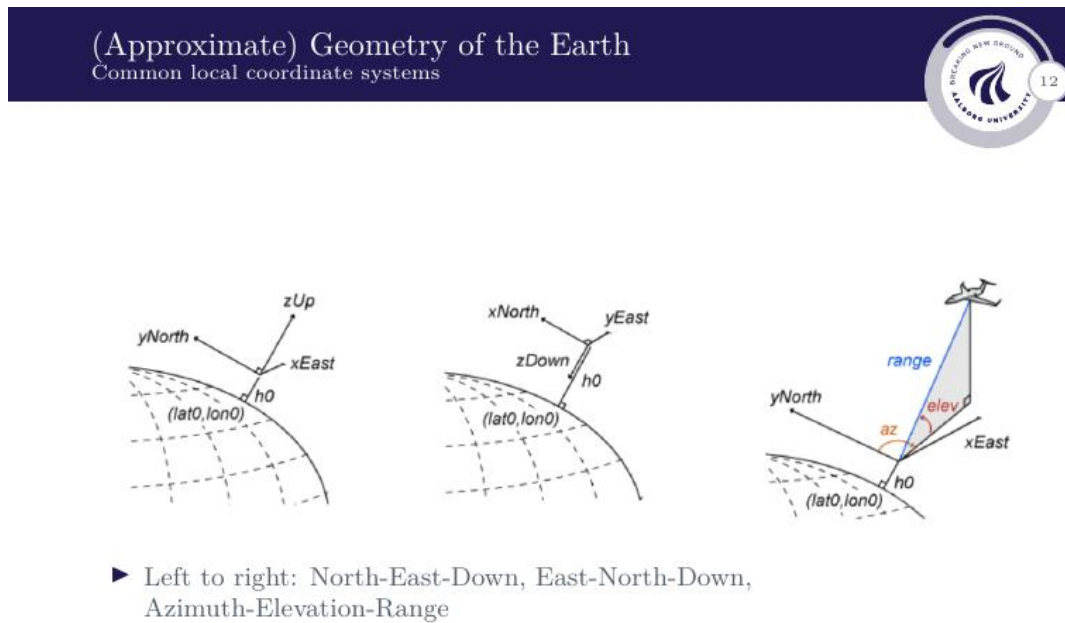


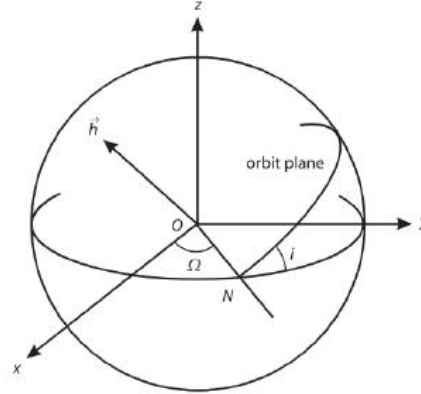
Figure 10: Three common local coordinate systems, all centred at a reference point $(\text{lat}_0, \text{lon}_0)$ on the surface. **Left: NED** (North-East-Down) — x North, y East, z Down. **Centre: ENU** (East-North-Up) — x East, y North, z Up. **Right: AER** (Azimuth-Elevation-Range) — describes a target by its compass direction, elevation angle above the horizon, and slant distance.

The three systems all describe the same local geometry, just with different axis choices:

- **NED (North-East-Down)**: used heavily in aviation and inertial navigation. z points *down* (into the Earth), which means altitude increases as z decreases — unintuitive but standard in many navigation systems.
- **ENU (East-North-Up)**: z points *up*, which feels more natural for ground-based use. Altitude increases as z increases.
- **AER (Azimuth-Elevation-Range)**: describes a target in polar-like terms — where to look (azimuth + elevation) and how far (range). Elevation is the angle *above* the horizon, so $\text{elevation} = 90^\circ - Z$ (zenith angle).

2.4 GPS Satellite Orbits

(Approximate) Geometry of the Earth Keplerian orbit



- Satellite position vector r in ECEF coordinates obeys:

$$\ddot{\mathbf{r}} = \begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} = -\frac{GM}{\|\mathbf{r}\|^3} \mathbf{r}$$

- Solution: Elliptic orbit in *orbital plane*.
- The orbital plane is perpendicular to the *angular momentum* $\mathbf{h} = \mathbf{r} \times \dot{\mathbf{r}}$.

Figure 11: Keplerian orbit. The satellite position vector r obeys Newton's gravitational equation. The solution is an ellipse in an orbital plane defined by the angular momentum vector $\mathbf{h} = \mathbf{r} \times \dot{\mathbf{r}}$. Key parameters: inclination i , right ascension of ascending node Ω , and argument of perigee ω .

A GPS satellite moves under Earth's gravity. Ignoring atmospheric drag and other small perturbations, its position vector $\mathbf{r} = [x, y, z]^\top$ obeys Newton's law:

$$\ddot{\mathbf{r}} = \begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} = -\frac{GM}{\|\mathbf{r}\|^3} \mathbf{r}$$

$\mathbf{r}$ and $\ddot{\mathbf{r}}$ are both 3D vectors with x, y, z components in the ECEF frame. The equation says the same thing in all three directions at once: gravity pulls the satellite back toward the origin (Earth's centre) with a strength that falls off with distance.

Symbol	Name	Physical meaning
r	Position vector	3D vector $[x, y, z]^T$ from Earth's centre to the satellite.
$\ddot{r}$	Acceleration vector	3D vector $[\ddot{x}, \ddot{y}, \ddot{z}]^T$ — the rate of change of velocity. Always points <i>back toward Earth's centre</i> (negative of r direction).
GM	Gravitational parameter	Product of gravitational constant G and Earth's mass M . $GM \approx 3.986 \times 10^{14} \text{ m}^3/\text{s}^2$.
$\ r\ $	Distance to centre	The further the satellite, the weaker the pull (falls off as $1/r^2$ after dividing by $\ r\ $).

What the equation is saying

The right-hand side $-(GM/\|r\|^3)r$ is a vector pointing *from the satellite back toward Earth's centre*, scaled by $GM/\|r\|^2$ (Newton's inverse-square gravity law). The satellite is always being pulled inward. At the right altitude and speed, this inward pull exactly balances the tendency to fly off in a straight line — the result is a stable orbit. GPS satellites orbit at $r \approx 26,561 \text{ km}$ where this balance gives an orbital period of ≈ 12 hours.

What is a Keplerian ellipse?

The solution to this equation is not a circle but an **ellipse** — a stretched circle with two *foci* instead of one centre. Earth sits at one focus.

- The **closest point** of the orbit to Earth is called the **perigee**. The satellite moves *fastest* here because gravity is strongest.
- The **farthest point** is called the **apogee**. The satellite moves *slowest* here.
- For GPS satellites the ellipse is nearly circular (very low eccentricity), so perigee and apogee are almost the same distance — the orbit barely looks stretched at all.

Why an ellipse and not a circle?

A perfect circle would require the satellite to be placed at exactly the right speed for its altitude — any small deviation and the orbit becomes slightly elongated. Kepler showed mathematically that gravity always produces conic sections (ellipses, parabolas, or hyperbolas) as the solution. For a satellite in a closed orbit it is always an ellipse. A circle is just a special case of an ellipse where both foci coincide at the centre — GPS orbits are close to this, but not perfectly circular.

The orbital plane and angular momentum.

The satellite traces its ellipse in a flat **orbital plane**. The angular momentum vector $h = r \times \dot{r}$ (position cross velocity) points perpendicular to this plane.

What “orbital plane perpendicular to angular momentum” means

Imagine spinning a ball on a string in a horizontal circle above your head. The ball moves in the horizontal plane. The angular momentum of the ball points *straight up*

— perpendicular to the horizontal plane it is moving in.

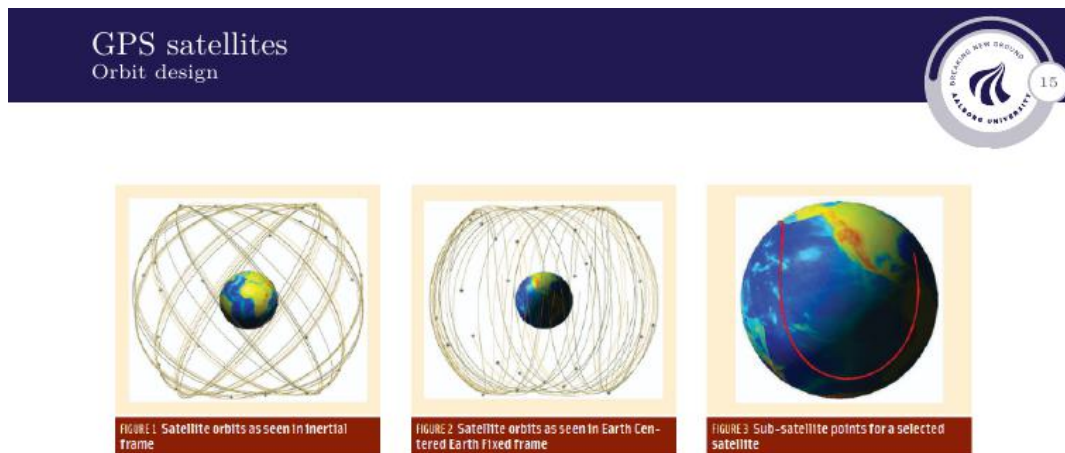
The same applies to a satellite. At every moment the satellite has a position vector r (pointing from Earth's centre outward) and a velocity vector $\dot{r}$ (pointing along its direction of travel). Both of these always lie *in* the orbital plane. The cross product $h = r \times \dot{r}$ is by definition perpendicular to both — so it sticks straight out of the orbital plane like a flagpole.

The key physics: in a pure gravitational field, angular momentum is **conserved** — it never changes direction or magnitude. This means the orbital plane never tilts. The satellite is permanently confined to the same flat plane, tracing the same ellipse over and over. The direction h points tells you which way the orbital plane is tilted relative to the equator — that is exactly what the inclination angle i and the ascending node angle Ω describe.

What the Keplerian orbit equation is saying — overall

The equation $\ddot{r} = -(GM/\|r\|^3) r$ says: the satellite accelerates toward Earth's centre with a strength that falls off as $1/r^2$. If the satellite is too close, gravity pulls it in faster than its orbital speed and it spirals inward. If it is too far, it drifts outward. At just the right altitude and speed these balance and the satellite keeps tracing the same ellipse indefinitely. GPS satellites orbit at $r \approx 26,561$ km where this balance gives an orbital period of ≈ 12 hours.

GPS constellation design.



K. Borre, "GPS EasySuite II", InsideGNSS June 2010

- ▶ 24+ satellites (a.k.a. "Space Vehicles")
- ▶ 6 orbital planes
- ▶ 55 degree inclination
- ▶ Elliptic (albeit nearly circular) orbits
- ▶ Average orbital radius: $r = 26,561$ km

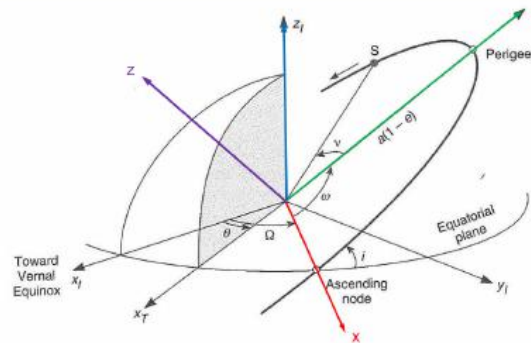
Figure 12: GPS constellation: 24+ satellites in 6 orbital planes inclined at 55° to the equator. Left: orbits as seen in the inertial frame (the Earth rotates inside the fixed orbital cage). Centre: orbits as seen in the ECEF frame (the orbital tracks appear to drift westward as the Earth rotates underneath). Right: the sub-satellite ground track for one satellite.

The GPS constellation is designed so that at least 4 satellites are visible from anywhere on Earth at all times:

- **24+ satellites** (officially called Space Vehicles, SVs).
- **6 orbital planes**, evenly spread around the equator (60° apart). Each plane holds 4 satellites.
- **55° inclination**: the orbital planes are tilted 55° from the equatorial plane, giving good coverage up to high latitudes.
- **Nearly circular orbits**, average radius $r = 26,561$ km.
- **Orbital period** $T \approx 11$ hr 58 min — exactly 2 revolutions per *sidereal day* (one Earth rotation relative to the stars). This means the satellites repeat the same ground track every day, shifted by 4 minutes relative to solar time.

The satellites transmit their precise orbital parameters (**broadcast ephemeris**) in the navigation message, so the receiver can compute where each satellite is at any given moment.

Computing satellite position in ECEF.



► Position in ECEF:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} \cos \Omega & -\sin \Omega & 0 \\ \sin \Omega & \cos \Omega & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos i & -\sin i \\ 0 & \sin i & \cos i \end{bmatrix} \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix}$$

$$= \begin{bmatrix} x' \cos \Omega - y' \sin \Omega \cos i \\ x' \sin \Omega + y' \cos \Omega \sin i \\ y' \sin i \end{bmatrix}$$

where $x' = r \cos(\nu + \omega)$, $y' = r \sin(\nu + \omega)$, and $z' = 0$.

Figure 13: Converting satellite orbital position to ECEF using two rotation matrices. First rotation by Ω aligns the orbital plane's reference direction with the ascending node. Second rotation by i tilts the orbital plane by the inclination angle.

Within its orbital plane the satellite position is simply:

$$x' = r \cos(\nu + \omega), \quad y' = r \sin(\nu + \omega), \quad z' = 0$$

where r is the current orbital radius, ν is the true anomaly (current angle from perigee), and ω is the argument of perigee.

Why is $z' = 0$ always — and do all satellites share the same coordinates?

The (x', y', z') coordinate system is a *local* frame that is defined to sit perfectly inside the orbital plane. x' points from Earth's centre toward perigee (the closest point of the orbit), y' is perpendicular to x' but still inside the same plane, and z' points *out* of the orbital plane entirely.

Do different satellites use different (x', y', z') coordinates? Yes — every satellite has its own orbital plane, and therefore its own (x', y', z') frame. The parameters Ω (which “clock position” around the Earth the plane is at) and i (how much it is tilted from the equator) are different for each satellite.

To convert this to ECEF two rotations are applied — first tilt the plane by inclination i ,

then rotate it to the correct longitude via Ω :

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \underbrace{\begin{bmatrix} \cos \Omega & -\sin \Omega & 0 \\ \sin \Omega & \cos \Omega & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{\text{rotate by } \Omega \text{ around } z} \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos i & -\sin i \\ 0 & \sin i & \cos i \end{bmatrix}}_{\text{rotate by } i \text{ around } x} \begin{bmatrix} x' \\ y' \\ 0 \end{bmatrix}$$

Multiplying out:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} x' \cos \Omega - y' \sin \Omega \cos i \\ x' \sin \Omega + y' \cos \Omega \cos i \\ y' \sin i \end{bmatrix}$$

Symbol	Name	Physical meaning
Ω	Right ascension of ascending node	The longitude (in the equatorial plane) where the satellite crosses from the southern to the northern hemisphere. Rotates the orbital plane to the correct “clock position” around the Earth.
i	Inclination	Angle between the orbital plane and the equatorial plane. $i = 0^\circ$ = equatorial orbit; $i = 90^\circ$ = polar orbit. GPS uses $i = 55^\circ$.
ω	Argument of perigee	Angle from the ascending node to the closest point of the orbit (perigee), measured in the orbital plane.
ν	True anomaly	Current angle of the satellite from perigee, measured in the orbital plane. This is the only parameter that changes as the satellite moves; all others are approximately constant.

What the two rotation matrices do

Start with the satellite sitting in a simple flat orbital plane — x' pointing toward perigee, y' perpendicular in the orbital plane, $z' = 0$.

The **first matrix** (rotation by i around the x -axis) *tilts* the orbital plane. Without this, all orbits would sit in the equatorial plane and never pass over high latitudes. GPS uses $i = 55^\circ$ to cover latitudes up to 55° north and south directly, and higher latitudes at lower elevations.

The **second matrix** (rotation by Ω around the z -axis) *spins* the already-tilted plane to the right longitude. With 6 orbital planes each separated by 60° in Ω , the satellites are spread evenly around the Earth.

Section 2 — Key Takeaways

- GPS needs **4 satellites** (not 3) because the receiver clock error τ is a 4th unknown alongside (x, y, z) . The measured distances are called pseudoranges because they

are offset by $c\tau$.

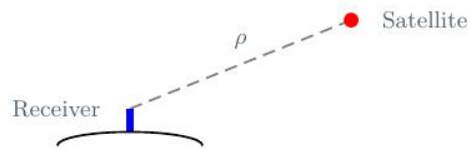
- **ECEF** is a Cartesian system rotating with the Earth. Origin at Earth's centre; z toward north pole; x toward Greenwich. Geographic (lat, lon) $\leftrightarrow$ ECEF via the spherical formula using $\cos \phi \cos \lambda$, $\cos \phi \sin \lambda$, $\sin \phi$.
- **Local frames** (NED, ENU, AER) are centred on the observer and describe relative directions (north/east/up or azimuth/elevation/range) — more intuitive than global ECEF for navigation.
- GPS satellites follow **Keplerian ellipses** governed by $\ddot{r} = -(GM/\|r\|^3)r$. 24+ SVs in 6 planes at 55° inclination and 26,561 km radius ensure ≥ 4 satellites visible worldwide.
- Satellite ECEF position is computed from orbital parameters using **two rotation matrices**: one tilt by inclination i , one spin by right ascension Ω .

Section 3 GPS Architecture & Signal Transmission

This section covers the practical mechanics of GPS: how a receiver actually measures a distance to a satellite, how the overall system is organised, what data the receiver spits out, how that data is packed inside the satellite signal, and how the receiver locks onto that signal in the first place.

3.1 Pseudorange — How the Distance Is Measured

Obtaining distance measurements



- ▶ Signal transmission time is encoded using an onboard atomic clock.
- ▶ Reception time is recorded by the receiver's own clock.
- ▶ Time-of-Flight (pseudorange) $\rho = c\Delta t$, where Δt is time difference between transmission and reception, and c is the speed of light.
- ▶ Includes clock errors because receiver clocks are imperfect.
- ▶ Satellite clock error is provided in the Navigation Message as a bit-wise polynomial.
- ▶ Receiver clock error is an unknown and must be estimated alongside receiver coordinates, giving a total of 4 unknowns (3 coordinates + 1 receiver clock error).

Figure 14: Pseudorange measurement. The satellite stamps the signal with its transmission time using its onboard atomic clock. The receiver records the arrival time with its own (imperfect) clock. The difference Δt , multiplied by c , gives the pseudorange ρ .

Every GPS satellite continuously broadcasts a signal that carries a precise timestamp: the exact moment (according to the satellite's atomic clock) when that signal left the satellite. Your receiver records the moment the signal arrives, using its own clock. The time difference multiplied by the speed of light gives an apparent range:

$$\rho = c \cdot \Delta t = c \cdot (T_{\text{received}} - T_{\text{transmitted}}^s)$$

Symbol	Name	Meaning
ρ	Pseudorange	The apparent satellite-to-receiver distance computed from the measured time difference. Called “pseudo” because it is contaminated by clock errors.
c	Speed of light	$\approx 3 \times 10^8$ m/s
T_{received}	Receiver clock time	The time the signal is recorded as arriving, according to the cheap quartz oscillator in your GPS receiver.
$T_{\text{transmitted}}^s$	Satellite clock time	The time the satellite stamped onto the signal when it left, according to the satellite’s onboard atomic clock.

Why “pseudo”range and not just range?

A true range would be $r = c \cdot \Delta t_{\text{true}}$, where Δt_{true} is the actual one-way signal travel time. In practice, two clock errors pollute this:

Satellite clock error δt^s : even atomic clocks drift slightly — typically less than a microsecond per day, but still enough to cause tens of metres of range error at the speed of light. The Control Segment tracks every satellite’s clock and uploads correction coefficients to each satellite. These coefficients are packed into the *Navigation Message* (a data stream inside the GPS signal) as a three-term polynomial:

$$\delta t^s = a_0 + a_1(t - t_0) + a_2(t - t_0)^2$$

Your receiver reads a_0, a_1, a_2 out of the Navigation Message and subtracts the correction — so satellite clock error is essentially a solved problem.

Receiver clock error τ : your receiver’s cheap quartz clock drifts by microseconds every second. There is no correction polynomial for this because the Control Segment does not know your receiver’s clock state. So τ is an unknown — and this is why GPS needs a **fourth satellite**. The pseudorange from satellite k is:

$$\rho_k = r_k + c\tau$$

where r_k is the true range and $c\tau$ is the same clock-offset error on every measurement. With four satellites you have four equations and four unknowns: x, y, z (receiver position) and τ (receiver clock error).

3.2 GPS System Architecture

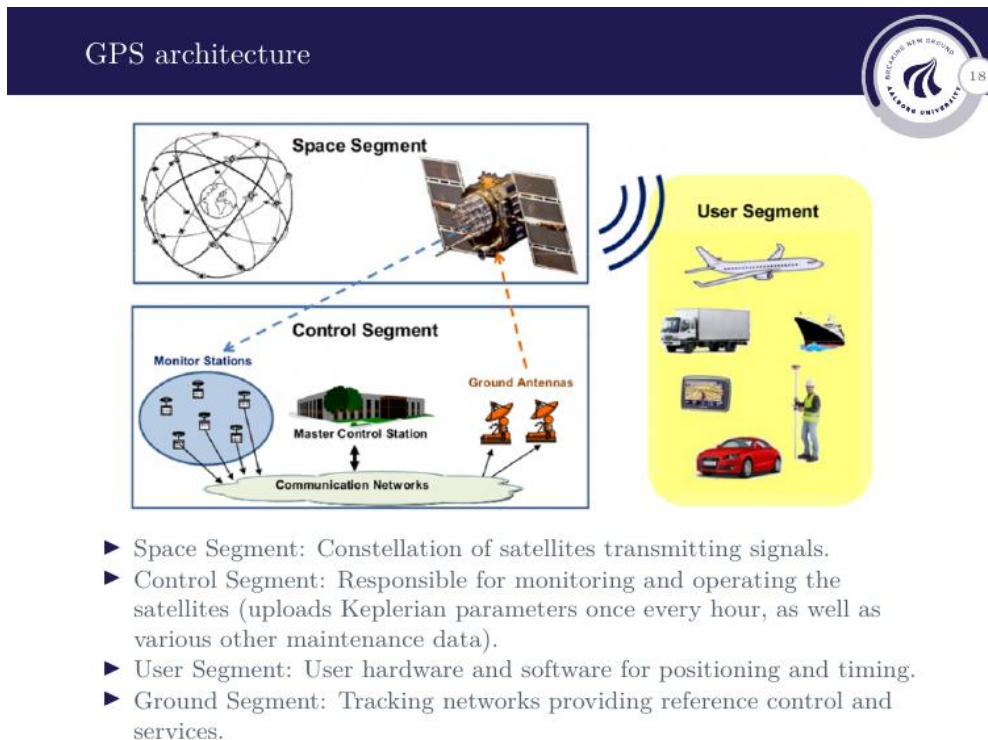


Figure 15: The four GPS segments: Space (satellites), Control (ground monitoring and command), User (any GPS receiver), and Ground (tracking reference networks).

GPS is not just a set of satellites — it is a system with four distinct layers:

Space Segment: The constellation of ≥ 24 GPS satellites orbiting at $\approx 20,200$ km altitude in six orbital planes. Each satellite continuously broadcasts two signals (L1 and L2) carrying a timing code and the Navigation Message.

What are L1 and L2?

L1 and L2 are the two radio frequencies GPS uses to broadcast its signal.

L1 (1575.42 MHz): the main civilian channel. It carries the *C/A code* (the pseudo-random ranging code every ordinary GPS receiver uses) plus the Navigation Message. Your phone, car satnav, and handheld GPS all use L1.

L2 (1227.60 MHz): a second frequency originally reserved for military use. Modern civilian receivers use a civilian component called *L2C*. The main reason for having *two* frequencies is to correct for **ionospheric delay**: the ionosphere (a layer of charged particles at 80–1000 km altitude) slows down radio waves, and the amount of slowing is different at different frequencies. By measuring how much L1 and L2 are delayed relative to each other, a dual-frequency receiver can calculate the exact ionospheric delay on that path and subtract it — improving accuracy from ~ 5 – 10 m to ~ 1 m without any ground infrastructure. Single-frequency receivers instead apply an approximate ionospheric model stored in the almanac (less accurate but good enough for everyday use).

Control Segment: A worldwide network of ground stations that tracks every satellite, maintains GPS time, and sends correction data up to each satellite roughly once per hour. It consists of:

- *Monitor stations* — passive receivers scattered around the globe that measure pseudo-ranges to every visible satellite and forward the data to the Master Control Station.
- *Master Control Station* — runs a Kalman filter to estimate each satellite’s precise orbital parameters and clock state, then generates the correction data (Keplerian parameters, clock polynomial).
- *Ground antennas* — uplink the corrections to the satellites.

How the Control Segment feeds the Navigation Message Yes — you have this right. The Control Segment is the origin of the clock correction polynomial (a_0, a_1, a_2) and the precise ephemeris that your receiver later reads from the Navigation Message. The chain is:

User Segment: Every device that receives GPS signals and computes a position — your phone, a car satnav, a plane’s avionics, a ship’s chart plotter. These receivers are *receive-only*: they never transmit anything back to the satellites.

Ground Segment: An umbrella term for all ground-based infrastructure that improves or monitors GPS beyond what the core Control Segment does. Think of it as the extra layer that turns “good enough for hiking” into “good enough for landing a plane.” It includes:

- *CORS (Continuously Operating Reference Stations)*: GPS receivers installed at precisely surveyed, fixed positions. Because their true position is known exactly, they can measure the gap between what GPS says and where they actually are — revealing the combined error from atmosphere, satellite orbits, and clocks at that location. This error is broadcast as a correction to nearby roving receivers (**Differential GPS / DGPS**), pushing accuracy from ~ 5 m down to ~ 0.5 – 2 m.
- *SBAS (Satellite-Based Augmentation Systems)*: networks of reference stations (e.g. WAAS in the USA, EGNOS in Europe) that compute differential corrections and integrity alerts, then rebroadcast them via geostationary satellites. Aviation uses SBAS because it provides a certified integrity signal — the system will alert you within seconds if GPS accuracy has degraded past a safe threshold.
- *Integrity monitoring*: independent ground networks that continuously verify each satellite is broadcasting correctly and flag unhealthy satellites for exclusion from safety-critical solutions.

3.3 GPS Satellite Hardware



Figure 16: A modern GPS Block IIF satellite. The large panels are solar arrays; the rectangular body houses the atomic clocks and signal electronics.

Each GPS satellite carries several redundant systems to ensure continuous operation over a 15-year design life:

- **Atomic clocks:** two rubidium and two caesium clocks onboard each satellite. Only one is active at a time; the Control Segment can command the satellite to switch to a backup if the primary clock degrades.

Both are **atomic clocks** — clocks that keep time by counting the oscillation of atoms rather than a vibrating crystal. An atom of a specific element always oscillates at exactly the same frequency, making atomic clocks extraordinarily stable. *Caesium* clocks are the most accurate: their oscillation frequency literally defines the SI second, and they drift less than 1 ns (one billionth of a second) per day. *Rubidium* clocks are slightly less stable but smaller and cheaper — they drift on the order of 1 μ s per day. Both are roughly 10^{10} times more stable than the quartz oscillator in your phone. The satellite carries both types as redundant backups: if one fails or drifts out of spec, the Control Segment switches the satellite to another.

- **Power:** large solar panels generate the main power; Ni-Cd batteries cover the ≈ 30 minutes per orbit when the satellite passes through Earth's shadow.
- **Health monitoring:** the Control Segment continuously assesses each satellite's signal quality and orbital accuracy. If a satellite drifts from its predicted orbit (due to solar radiation pressure, lunar gravity, or atmospheric drag on low passes), the Control Segment can command small thruster burns to restore the correct slot.
- **Geometry management:** satellites can be repositioned to maintain good geometric

spread across the constellation, which directly affects positioning accuracy (covered in Section 4).

Think of it this way: imagine trying to locate yourself on a map using three friends standing at known positions, but all three happen to be clustered in the same direction (e.g. all three are northeast of you). Each friend gives you a distance, but because they are all roughly in the same direction, the three circles you draw nearly overlap — a tiny measurement error shifts the intersection by a huge amount. Now spread those friends out evenly around you (north, south-east, west) and the circles cross at a sharp, well-defined point — a small error barely moves the answer.

The same applies to GPS satellites. If the four visible satellites happen to be clustered in one patch of sky, the geometry is poor and position error is amplified — quantified by a number called **DOP (Dilution of Precision)**; a high DOP means bad geometry. The six orbital planes are deliberately designed so there are always enough satellites spread across the whole sky from anywhere on Earth. If a satellite fails or is taken offline for maintenance, the Control Segment may command neighbouring satellites to maneuver slightly within their slots to fill the gap and restore good geometric coverage.

3.4 GPS Data Output — NMEA Sentences

GPS data output string

Examples of GPS “sentences”



```
$GPRMC,044840.00,A,3401.21044,N,11824.67722,W,0.165,,010621,,,D*60
$GPVTG,,T,,M,0.165,N,0.306,K,D*21
$GPGGA,044840.00,3401.21044,N,11824.67722,W,2,07,1.34,29.5,M,-32.9,M,
$GPGSA,A,3,21,32,46,27,08,22,10,,,,,3.07,1.34,2.76*0F
$GPGSV,3,1,12,01,18,319,17,08,27,257,20,10,44,068,33,18,00,142,*73
$GPGSV,3,2,12,21,39,313,23,22,20,296,14,23,16,089,,27,22,221,32*7A
$GPGSV,3,3,12,31,28,168,20,32,70,025,31,46,49,199,30,51,49,161,*7E
$GPGLL,3401.21044,N,11824.67722,W,044840.00,A,D*7F
```

- ▶ GPGGA: Global positioning system fix data (time, position, fix type data)
- ▶ GPGLL: Geographic position, latitude, longitude
- ▶ GPVTG: Course and speed information relative to the ground
- ▶ GPRMC: Time, date, position, course, and speed data
- ▶ GPGSA: GPS receiver operating mode, satellites used in the position solution, and DOP values.
- ▶ GPGSV: The number of GPS satellites in view, satellite ID numbers, elevation, azimuth, and SNR values.

See e.g., <https://www.redhat.com/en/blog/architects-guide-gps-and-gps-data-formats>

Figure 17: A typical burst of NMEA sentences output by a GPS receiver once per second. Each line is a self-contained, human-readable ASCII sentence.

A GPS receiver outputs its results as a stream of plain-text **NMEA 0183 sentences** — comma-separated ASCII lines, one per data type, updated once per second. Each sentence starts with a \$ and a five-letter identifier.

What each NMEA sentence type contains

\$GPGGA — Global Positioning System Fix Data. The most important sentence: time of fix (UTC), latitude, hemisphere (N/S), longitude, hemisphere (E/W), fix quality, number of satellites used, HDOP, altitude above mean sea level, and geoid height. This is the sentence you parse if you want position and quality.

\$GPGLL — Geographic position: latitude and longitude only, with a validity flag. Minimal version of GPGGA for applications that only need coordinates.

\$GPVTG — Course over ground and speed over ground, expressed in both degrees-true/degrees-magnetic and km/h/knots. Tells you the direction and speed of movement, not the position.

\$GPRMC — Recommended Minimum Specific GNSS Data. A single sentence combining time, date, position, speed, and course — the single sentence that older devices often used as the sole GPS output.

\$GPGSA — GPS Receiver Operating Mode and DOP. Lists which satellites are included in the current position solution and gives dilution-of-precision (DOP) values for that geometry.

\$GPGSV — GPS Satellites in View. Reports the satellite ID, elevation angle, azimuth, and signal-to-noise ratio for every satellite the antenna can currently see, even those not used in the position fix.

GPS data output string			
GPGGA, Global Positioning System Fix Data			
\$GPGGA,170241.00,3401.21189,N,11824.67797,W,1,06,2.14,71.6,M,-32.9,M,			
Field	Name	Data	Description
0	Type ID	\$GPGGA	GGA protocol header
1	Time	170241	17:02:41 UTC
2	Latitude	3401.21189	ddmm.mmmm format, converts to 34.020196 or 34d 1' 12.706" N
3	Hemisphere	N	N = North, S = South
4	Longitude	11824.67797	dddmm.mmmm format, converts to -118.41129833 or 118d 24' 48.673" W
5	Hemisphere	W	W = West, E = East
6	fix type	1	Data is from a GPS fix
7	No. sats	06	6 Satellites are in view
8	HDOP	2.14	Relative accuracy of horizontal position
9	Alt.	71.6	71.6 meters above mean sea level
10	Alt. unit	M	M = meters
11	Geoid height	-32.9	-32.9 meters above WGS84 ellipsoid
12	Height unit	M	M = meters
13	Time		Time since last DGPS update (no last upc
14	DGPS ID		DGPS station id (no station ID)
15	Checksum	*57	Check for transmission errors

Figure 18: Field-by-field breakdown of a \$GPGGA sentence. Latitude is in ddmm.mmmm format; to get decimal degrees divide the last four digits by 60. Altitude is above mean sea level; geoid height corrects to WGS84 ellipsoid.

The \$GPGGA sentence is the one most software libraries parse first. Taking the example on the slide: \$GPGGA,170241.00,3401.21189,N,11824.67797,W,1,06,2.14,71.6,M,-32.9,M,*57

- Time 170241.00 = 17:02:41 UTC.
- Latitude 3401.21189,N: the first two digits are whole degrees (34°), the rest are decimal minutes (01.21189 minutes north), giving $34 + 01.21189/60 = 34.02020^\circ$ N.
- Longitude 11824.67797,W: $118^\circ + 24.67797/60$ minutes west = 118.4113° W (Los Angeles area).
- Fix type 1 = standard GPS fix (not differential).
- 06 satellites used in this solution.
- HDOP 2.14: horizontal dilution of precision — a unitless multiplier on the nominal range error (smaller is better; ≤ 2 is considered good).
- Altitude 71.6 M above mean sea level.
- Geoid height -32.9 M: the WGS84 ellipsoid is 32.9 m *above* mean sea level at this location, so height above ellipsoid = $71.6 + (-32.9) = 38.7$ m.
- Checksum *57: XOR of all characters between \$ and *, for error detection.

3.5 Navigation Message Structure



The navigation message consists of 30-second frames 1,500 bits long, divided into five 6-second subframes of ten 30-bit words each:

Sub-frame	Word	Description
1	1-2	Telemetry and handover words (TLM and HOW)
	3-10	Satellite clock, GPS time relationship
2-3	1-2	Telemetry and handover words (TLM and HOW)
	3-10	Ephemeris (precise satellite orbit)
4-5	1-2	Telemetry and handover words (TLM and HOW)
	3-10	Almanac component (satellite network synopsis, error correction)

- *Ephemeris* is precise orbital information for the transmitting satellite.
- *Almanac* consists of coarse orbit and status information for each satellite in the constellation, an ionospheric model, and information to relate GPS derived time to Coordinated Universal Time (UTC).

Figure 19: Structure of the GPS Navigation Message. Each 30-second frame carries 1,500 bits in five 6-second subframes. The payload splits into clock data, precise ephemeris, and almanac.

Every GPS satellite broadcasts a continuous **Navigation Message** at 50 bits per second. The message is organised into frames:

- One **frame** = 30 seconds = 1,500 bits.
- Each frame is split into **five subframes** of 6 seconds each.
- Each subframe contains **ten 30-bit words**.
- Every subframe starts with Telemetry (TLM) and Handover (HOW) words that let the receiver synchronise to the frame boundary.

What is inside each subframe?

Subframe 1 (satellite clock data): the three-term polynomial (a_0, a_1, a_2) that corrects the satellite's atomic clock to GPS system time, plus the GPS week number and satellite health flags. Your receiver reads this and removes the satellite clock error from every pseudorange.

Subframes 2–3 (ephemeris): the precise Keplerian orbital parameters for this particular satellite, including:

- Ω — right ascension of the ascending node: which “clock position” around the Earth the orbital plane is currently at (i.e. how the plane is rotated around Earth's spin axis).
- i — inclination: the tilt of the orbital plane relative to the equator (55° for GPS).
- ω — argument of perigee: where the closest point of the orbit (perigee) sits within the orbital plane.
- ν — true anomaly: the satellite's current angle from perigee, measured in the orbital plane. This is the only parameter that changes rapidly as the satellite moves; all others evolve slowly.
- *Eccentricity* — how elliptical the orbit is (0 = perfect circle; GPS orbits are nearly circular at $e \approx 0.01$).
- *Semi-major axis* a — the size of the orbit ($a \approx 26,561$ km for GPS).

Together these six numbers, plugged into the two-rotation-matrix formula from Section 2.4, let the receiver compute the satellite's exact ECEF (x, y, z) position at any moment. The ephemeris also includes small correction terms for real-world perturbations (solar radiation pressure, lunar/solar gravity pulls) that make the orbit deviate slightly from a perfect Keplerian ellipse. Ephemeris data is valid for about 2 hours; after that the satellite has drifted enough that a fresh upload from the Control Segment is needed.

Subframes 4–5 (almanac): a lower-precision version of the same orbital data, but for *every* satellite in the constellation (not just this one). Each satellite's almanac entry contains a coarse Ω , i , eccentricity, and semi-major axis — accurate enough to predict roughly where a satellite will be, but not accurate enough to compute a precise pseudorange. The almanac also carries:

- An *ionospheric correction model* (the coefficients for the Klobuchar model) — used by single-frequency receivers to estimate and partially remove ionospheric delay.
- A *GPS-to-UTC time offset* — because GPS time and Coordinated Universal Time differ by a whole number of leap seconds (GPS does not insert leap seconds; UTC does).

Almanac data stays valid for up to several weeks because orbital planes change only very slowly.

The **difference between ephemeris and almanac** is precision and scope. Ephemeris is centimetre-level accurate but only describes the one satellite you are currently listening to, and expires in 2 hours. Almanac is metre-to-kilometre-level accurate but covers all 30+ satellites and stays valid for weeks. The practical payoff: when you switch on a GPS receiver that has been off for a few days (*cold start*), it has no idea which satellites are above the horizon right now. If it still has a valid almanac stored in memory, it can compute a prediction of which satellites to expect and point its search at those — finding a signal lock in 30–60 seconds instead of scanning blindly for 10+ minutes.

3.6 Signal Transmission & Reception

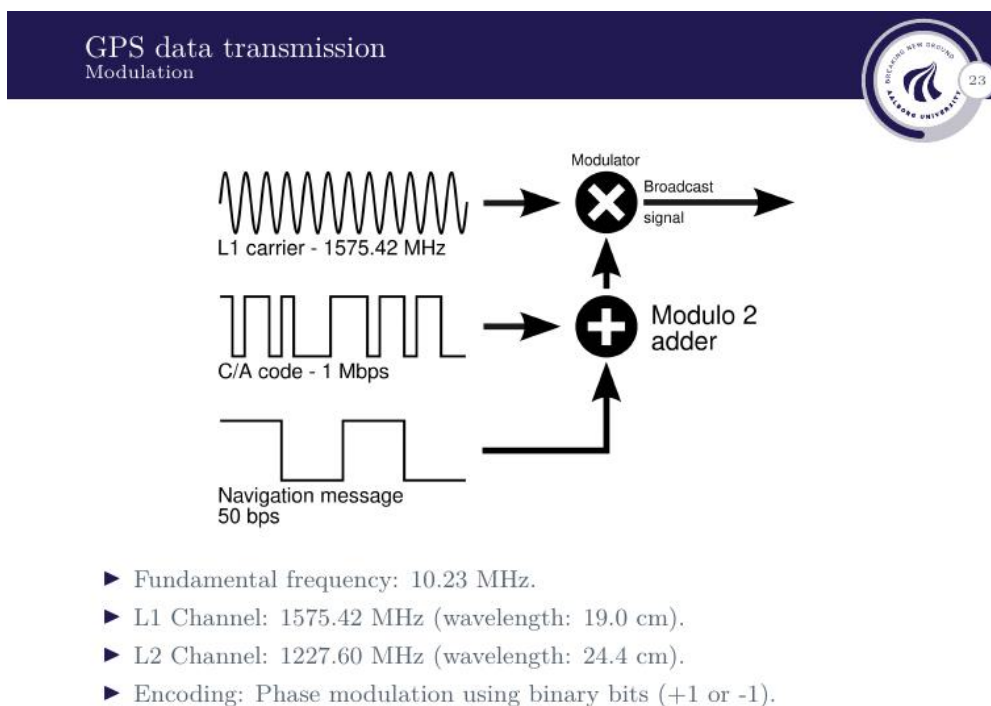


Figure 20: GPS signal structure. The L1 carrier (1575.42 MHz) is phase modulated by the modulo-2 sum of the C/A ranging code (1 Mbps) and the Navigation Message (50 bps).

Three layers, three purposes

The GPS broadcast signal is built in three distinct layers, each doing a different job. All three derive from a single master oscillator at $f_0 = 10.23$ MHz — the atomic clock

frequency — so they are all perfectly synchronised to the same source.

Layer 1 — the L1 carrier ($154 \times f_0 = 1575.42 \text{ MHz}$): A pure radio sine wave. On its own it carries no information — it is just the vehicle that makes everything propagate through 20 000 km of space. A data signal at 1 MHz cannot travel that distance; it needs to ride on a high-frequency carrier, exactly as music rides on an FM radio station at e.g. 100 MHz. (L2 at $120 \times f_0 = 1227.60 \text{ MHz}$ serves the same purpose on a second frequency for ionospheric correction.)

Layer 2 — the C/A code ($f_0/10 = 1.023 \text{ Mbps}$): A pseudorandom sequence of 1023 chips repeating every millisecond. It serves two roles simultaneously:

- *Satellite ID (CDMA):* every satellite broadcasts a different C/A code. All 30+ satellites share the same L1 frequency but your receiver can lock onto any individual one by searching for its unique code pattern.
- *Ranging:* because the code changes ≈ 1 million times per second, autocorrelating it against a local replica gives microsecond-precision timing — the source of the pseudorange measurement. Timing resolution is $\approx 1/f_{C/A} \approx 1 \mu\text{s} \approx 300 \text{ m}$, refined to $\sim 1 \text{ ns}$ ($\sim 30 \text{ cm}$) by carrier-phase tracking.

Layer 3 — the Navigation Message (50 bps): The actual data payload: clock correction polynomial, ephemeris, almanac. It changes far too slowly (50 times per second) to be useful for nanosecond timing, but that is fine — timing comes from the C/A code, not from the data bits.

Why XOR ($\oplus$) to combine C/A and Navigation Message?

You need to transmit both layers simultaneously on the same signal. XOR (modulo-2 addition, $\oplus$) is used because:

In ± 1 notation XOR is simply *multiplication*:

$$\text{Combined} = C/A \times \text{Nav}$$

- When Nav bit = +1: Combined = $+1 \times C/A = C/A$ (chips pass through unchanged).
- When Nav bit = -1: Combined = $-1 \times C/A = -C/A$ (every chip flipped — polarity inverted).

Why this specific operation?

1. *Preserves autocorrelation.* A flipped C/A code ($-C/A$) has exactly the same autocorrelation peak as the original ($+C/A$) — the receiver locks onto the timing regardless of what the Nav bit is. The C/A code still does its ranging job even while carrying the Nav data inside its polarity.
2. *Self-inverse recovery.* XOR is its own inverse: Combined $\oplus C/A = \text{Nav}$. Once the receiver has locked onto the C/A code and knows what pattern to expect, it XORs it out of the received signal — and the Nav bits are what remains.
3. *No extra bandwidth.* The combined signal still runs at the C/A chip rate (1 Mbps).

No extra frequencies or bandwidth are needed to carry the 50 bps Nav data — it is hidden inside the polarity of the C/A chips.

The final step — BPSK onto the carrier: The combined ± 1 sequence still needs to travel through space. This is done by *phase-modulating* the L1 carrier: when the combined bit is $+1$ the carrier phase is left unchanged; when it is -1 the carrier phase is flipped by 180° . This is called BPSK (Binary Phase-Shift Keying).

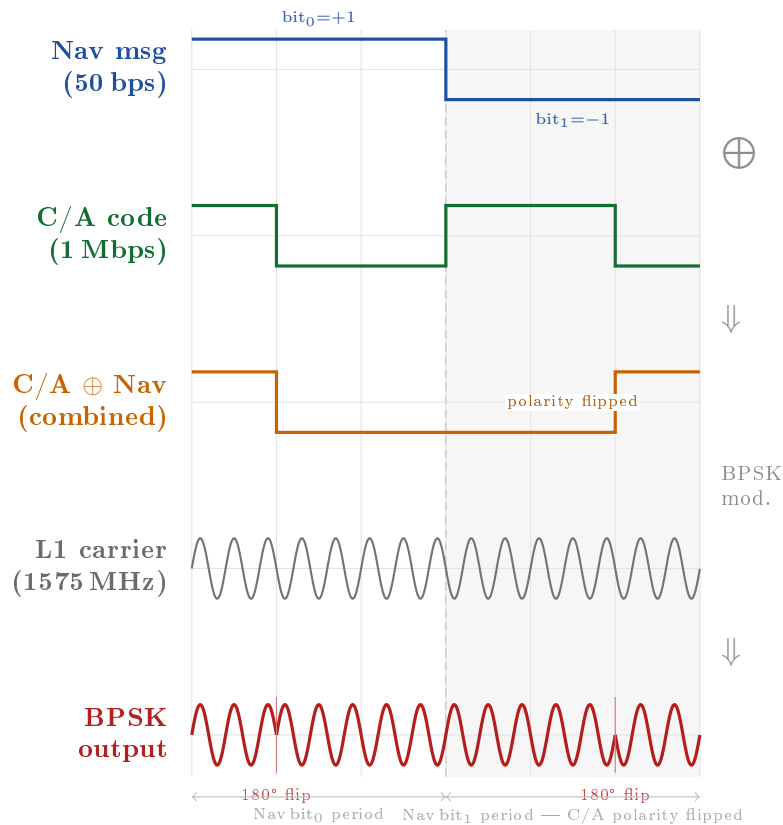


Figure 21: How the GPS signal is built up layer by layer. **Blue:** Navigation Message (slow data bits, 50 bps) — long rectangles showing $\text{bit}_0 = +1$ then $\text{bit}_1 = -1$. **Green:** C/A ranging code (fast pseudorandom chips, 1 Mbps). **Orange:** XOR-combined signal — identical to C/A during bit_0 , but with every chip polarity flipped during bit_1 (shaded region). **Grey:** L1 carrier sine wave (schematic; real frequency $\approx 1540 \times$ the C/A chip rate). **Red:** final BPSK broadcast signal — the carrier with 180° phase flips wherever the combined signal is -1 . The receiver decodes the signal in two steps: (1) autocorrelate to find C/A lock and extract timing; (2) track polarity flips to read Nav bits.

Why a pseudorandom C/A code?

The C/A code is a specific sequence of 1023 chips (bits) chosen for its excellent autocorrelation properties: when you multiply the sequence by a time-shifted version of itself and sum the result, you get a large spike only when the two copies are perfectly aligned, and near-zero otherwise. This is the mechanism the receiver uses to measure time-of-flight.

Every satellite broadcasts a *different* C/A code (there are 1023 distinct codes, one assigned to each satellite ID). This means all satellites can transmit on the same L1 frequency simultaneously without interfering — a technique called **CDMA** (Code Division Multiple Access). Your receiver knows which codes belong to which satellites and can tune into each one independently.

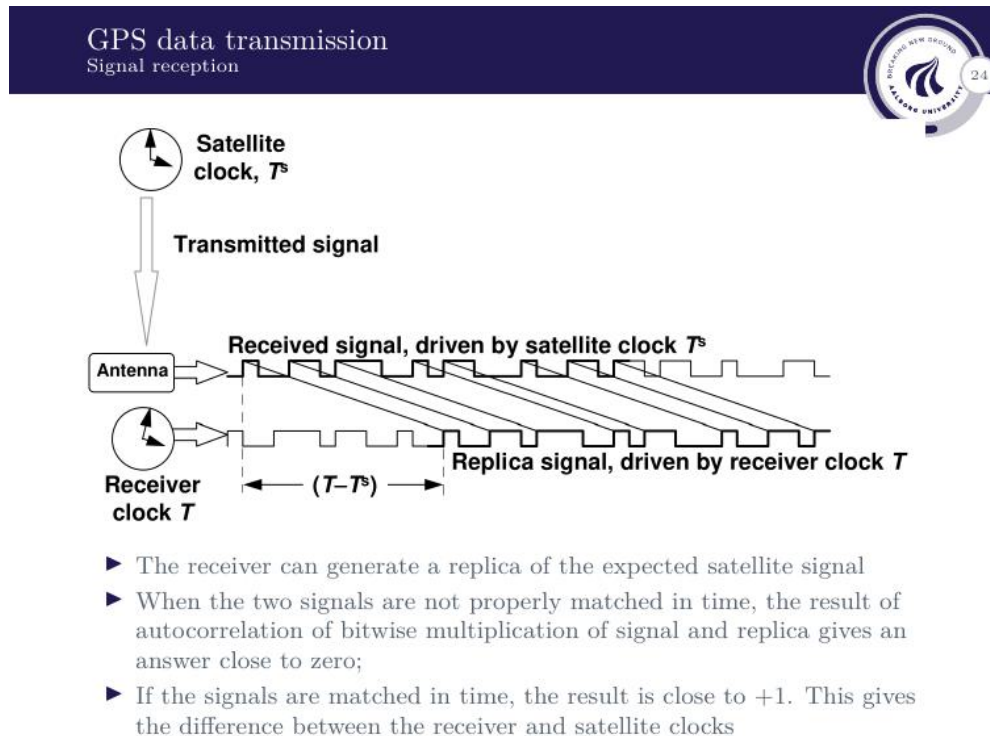


Figure 22: Signal reception by autocorrelation. The receiver generates a local replica of the C/A code, driven by its own clock T . It slides the replica in time until the autocorrelation with the incoming signal peaks at +1. The time shift at the peak is $T - T^s$, the clock difference used to compute the pseudorange.

The receiver cannot simply look at a waveform and read off a timestamp — the signal has no pulse or edge to identify. Instead it uses **autocorrelation**:

1. The receiver generates a local **replica** of the satellite's C/A code, running on its own (imperfect) clock T .
2. It multiplies the received signal by the replica bit by bit and sums the result.

How the receiver measures arrival time

The receiver does *not* receive a pre-made copy of the C/A code — it **generates one locally from scratch**. Every GPS receiver contains the algorithm and the code table for every satellite, so it can reproduce any satellite's 1023-chip sequence internally at any time.

The received signal has the satellite's C/A code embedded in it, but that code started being transmitted at the satellite's clock time T^s . The receiver's local replica starts from

its own clock time T . Those two clocks are offset by an unknown amount $\Delta t = T - T^s$, so the two copies of the code are *shifted* relative to each other by exactly Δt .

The receiver tries different lag values k — asking “what if I delay my replica by k chips?” — and for each k computes the **cross-correlation**:

$$R(k) = \frac{1}{N} \sum_{i=0}^{N-1} x_i \cdot y_{i-k}$$

where x_i is the received chip and y_{i-k} is the local replica delayed by k chips. This is exactly the lag- k correlation from signal processing:

- $k \neq \Delta t$: the two pseudorandom sequences are misaligned. Half the products are $(+1) \times (-1) = -1$ and half are $(+1) \times (+1) = +1$ — they nearly cancel, so $R(k) \approx 0$.
- $k = \Delta t$: the sequences line up perfectly. Every chip multiplies its own copy, so every product = +1, giving $R(\Delta t) = +1$.

There is a single sharp spike in R at exactly one lag value and near-zero everywhere else. The receiver scans through trial lags (or uses a *delay-locked loop* to continuously track the peak) until it finds that spike. The lag k where $R(k) = 1$ is the answer:

$$\Delta t = k \quad \Rightarrow \quad \rho = c \Delta t$$

The autocorrelation is therefore exactly what tells the receiver: *“the two signals are fully correlated at lag k — so the signal must have taken time k to travel from the satellite to me.”*

Connection to Games-on-Track chirp (Section 1)

This is the same cross-correlation timestamping principle used by the radio chirp system. Instead of a swept frequency, GPS uses a pseudorandom binary code. Both work on the same idea: transmit a known pattern, generate a local copy, slide it in time until the two match — the required shift is the travel time. The C/A code’s 1023-chip pseudorandom structure is chosen specifically because it produces a very sharp, unambiguous autocorrelation spike with near-zero sidelobes, making the timing measurement clean and precise.

Section 3 — Key Takeaways

- The measured quantity is the **pseudorange** $\rho = c\Delta t = r + c\tau$, where r is true range and τ is receiver clock error. Satellite clock error is correctable from the Navigation Message; τ is solved alongside (x, y, z) using four satellites.
- GPS has four segments: **Space** (satellites), **Control** (ground monitoring, orbit/clock corrections), **User** (receivers), **Ground** (tracking reference networks).
- Each satellite carries redundant **atomic clocks**; the Control Segment monitors health and uploads Keplerian updates hourly.
- Receivers output **NMEA sentences** (e.g. \$GPGGA) — plain ASCII with position, time, altitude, DOP, and satellite count.

- The **Navigation Message** (50 bps) carries clock corrections (subframe 1), precise *ephemeris* for this satellite (subframes 2–3), and coarse *almanac* for all satellites (subframes 4–5).
- The receiver measures pseudorange via **autocorrelation**: it slides a local C/A code replica until it aligns with the incoming signal; the required time shift is $\Delta t = T - T^s$. Different satellites use different C/A codes on the same frequency (**CDMA**).

Section 4 GPS Position Estimation

We now have all the pieces: satellites with known ECEF positions, pseudorange measurements from each, and the understanding that those measurements are corrupted by the receiver clock error τ . This section shows how to go from a set of pseudoranges to an estimated receiver position — and how to quantify how uncertain that estimate is.

4.1 The Pseudorange Observation Equation

GPS pseudorange



- Observation equation for distance to satellite S :

$$P_S = (T - T_S)c$$

- Incorporating clock biases $T = t + \tau$ (receiver) and $T_S = t_S + \tau_S$ (satellite):

$$P_S = (t + \tau)c - (t_S + \tau_S) = \rho(t, t_S) + c\tau - c\tau_S$$

- Geometric distance between receiver and satellite:

$$\rho(t, t_S) = \sqrt{(x_S(t_S) - x(t))^2 + (y_S(t_S) - y(t))^2 + (z_S(t_S) - z(t))^2}$$

- $(x_S(t_S), y_S(t_S), z_S(t_S))$ and τ_S are transmitted from the satellite; that leaves 4 unknowns: $(x(t), y(t), z(t))$ and τ .
- Ignores atmospheric delays and relativity for simplicity.

Figure 23: Full pseudorange model incorporating both the receiver clock bias τ and the satellite clock bias τ_S . The satellite transmits its position and τ_S ; the receiver solves for its own position and its unknown τ .

The raw measurement is the pseudorange P_S , computed from the time difference between transmission and reception:

$$P_S = (T - T_S)c$$

Both clocks deviate from true GPS time. Substituting $T = t + \tau$ (receiver) and $T_S = t_S + \tau_S$ (satellite):

$$P_S = [(t + \tau) - (t_S + \tau_S)]c = \underbrace{c(t - t_S)}_{\rho(t, t_S)} + c\tau - c\tau_S$$

so the pseudorange is:

$$P_S = \rho(t, t_S) + c\tau - c\tau_S$$

Symbol	Name	Meaning
P_S	Pseudorange	The measured apparent distance: raw time difference $\times c$. “Pseudo” because it includes clock offsets.
$\rho(t, t_S)$	True geometric range	Actual Euclidean distance from receiver to satellite: $\sqrt{(x_S - x)^2 + (y_S - y)^2 + (z_S - z)^2}$.
τ	Receiver clock error	Offset of the receiver’s quartz oscillator from true GPS time. Unknown — must be estimated alongside position.
τ_S	Satellite clock error	Offset of the satellite’s atomic clock from true GPS time. Known — provided in the Navigation Message as a polynomial; $c\tau_S$ is subtracted before further processing.
t	True reception time	True GPS time when signal arrives at receiver.
t_S	True transmission time	True GPS time when satellite sent the signal.

What are the unknowns?

After correcting the known satellite clock term, the corrected pseudorange is:

$$\tilde{P}_S = P_S + c\tau_S = \rho(t, t_S) + c\tau$$

The satellite position (x_S, y_S, z_S) at time t_S is known from the ephemeris (Section 2.4), so ρ contains three unknowns: the receiver coordinates $x(t)$, $y(t)$, $z(t)$. Plus the fourth unknown τ , the receiver clock error.

4 unknowns, 1 equation per satellite — need at least 4 satellites for a unique solution. This is the same “why 4 satellites?” conclusion from Section 2.1, now seen as a proper equation system.

Atmospheric delays (ionospheric and tropospheric signal slowing) and relativistic effects are real but are omitted here for simplicity; operational receivers apply models to correct for both.

4.2 Linearising for Multiple Satellites

GPS trilateration

Linear formulation



- For $m \geq 4$ satellites in view:

$$P_1 = \sqrt{(x_1(t_1) - x(t))^2 + (y_1(t_1) - y(t))^2 + (z_1(t_1) - z(t))^2} + c\tau - c\tau_1$$

$$P_2 = \sqrt{(x_2(t_2) - x(t))^2 + (y_2(t_2) - y(t))^2 + (z_2(t_2) - z(t))^2} + c\tau - c\tau_2$$

$\vdots$

$$P_m = \sqrt{(x_m(t_m) - x(t))^2 + (y_m(t_m) - y(t))^2 + (z_m(t_m) - z(t))^2} + c\tau - c\tau_m$$

- Linearize around (x_0, y_0, z_0, τ_0) to obtain

$$b = A\hat{\xi} + \nu$$

where

$$\hat{\xi} = \begin{bmatrix} x - x_0 \\ y - y_0 \\ z - z_0 \\ \tau - \tau_0 \end{bmatrix}, \quad b = \begin{bmatrix} P_1 - P_0 \\ P_2 - P_0 \\ \vdots \\ P_m - P_0 \end{bmatrix}, \quad A = \begin{bmatrix} \frac{\partial P_1}{\partial x} & \frac{\partial P_1}{\partial y} & \frac{\partial P_1}{\partial z} & \frac{\partial P_1}{\partial \tau} \\ \frac{\partial P_2}{\partial x} & \frac{\partial P_2}{\partial y} & \frac{\partial P_2}{\partial z} & \frac{\partial P_2}{\partial \tau} \\ \vdots & \vdots & \vdots & \vdots \\ \frac{\partial P_m}{\partial x} & \frac{\partial P_m}{\partial y} & \frac{\partial P_m}{\partial z} & \frac{\partial P_m}{\partial \tau} \end{bmatrix}$$

and ν contains noise terms.

Figure 24: With $m \geq 4$ satellites the m pseudorange equations are stacked and linearised around an initial guess, giving the linear system $b = A\hat{\xi} + \nu$.

With m satellites the corrected equations are:

$$\tilde{P}_k = \rho_k(x, y, z, t_k) + c\tau, \quad k = 1, \dots, m$$

Each ρ_k is a square root of squared differences — the system is **nonlinear** in the unknowns (x, y, z, τ) .

The standard approach: linearise around an initial guess (x_0, y_0, z_0, τ_0) using a first-order Taylor expansion, exactly as in the Jacobian linearisation from Lecture 7. Let $P_{0,k}$ be the predicted pseudorange to satellite k at the initial guess. Define the **correction vector** $\hat{\xi}$ and **measurement-difference vector** b :

$$\hat{\xi} = \begin{bmatrix} x - x_0 \\ y - y_0 \\ z - z_0 \\ \tau - \tau_0 \end{bmatrix}, \quad b = \begin{bmatrix} \tilde{P}_1 - P_{0,1} \\ \tilde{P}_2 - P_{0,2} \\ \vdots \\ \tilde{P}_m - P_{0,m} \end{bmatrix}$$

Stacking all m equations gives:

$$b = A\hat{\xi} + \nu$$

Here b contains the *known* numbers — what you measured minus what you predicted — and A is the $m \times 4$ Jacobian (defined below). ν is the **fit residual** — the leftover mismatch after subtracting the model's prediction $A\hat{\xi}$ from b :

$$\nu = b - A\hat{\xi}$$

To be clear on the difference:

- $b_k = \tilde{P}_k - P_{0,k}$: how far the *observed* pseudorange is from what the initial guess predicted. This is *computed from data* and is the input to the problem.
- $\nu_k = b_k - (A\hat{\xi})_k$: how far the fitted model still misses the data after the best $\hat{\xi}$ is found. This is what the fit *cannot explain* — due to measurement noise and the truncated higher-order Taylor terms.

In a perfect, noiseless, linear world $\nu = \mathbf{0}$ and $b = A\hat{\xi}$ exactly. In practice $\nu \neq \mathbf{0}$; least squares finds the $\hat{\xi}$ that makes $\|\nu\|^2$ as small as possible. The $m \times 4$ Jacobian A is:

$$A = \begin{bmatrix} \frac{\partial P_1}{\partial x} & \frac{\partial P_1}{\partial y} & \frac{\partial P_1}{\partial z} & \frac{\partial P_1}{\partial \tau} \\ \vdots & \vdots & \vdots & \vdots \\ \frac{\partial P_m}{\partial x} & \frac{\partial P_m}{\partial y} & \frac{\partial P_m}{\partial z} & \frac{\partial P_m}{\partial \tau} \end{bmatrix}$$

What are the partial derivatives in A ?

The first three columns of each row are the partial derivatives of ρ_k with respect to receiver position:

$$\frac{\partial P_k}{\partial x} = -\frac{x_S^{(k)} - x_0}{\rho_{0,k}}, \quad \frac{\partial P_k}{\partial y} = -\frac{y_S^{(k)} - y_0}{\rho_{0,k}}, \quad \frac{\partial P_k}{\partial z} = -\frac{z_S^{(k)} - z_0}{\rho_{0,k}}$$

These are the *direction cosines* from the initial receiver guess to satellite k — the components of the unit vector pointing from receiver toward the satellite. Physically: moving the receiver by a small step in x changes the range to satellite k only by the projection of that step onto the receiver-to-satellite direction.

The fourth column is simply c for every satellite:

$$\frac{\partial P_k}{\partial \tau} = c$$

because a clock offset $\delta\tau$ adds the same $c\delta\tau$ to every pseudorange, regardless of which satellite.

So each row of A looks like:

$$\left[-\frac{x_S^{(k)} - x_0}{\rho_{0,k}} \quad -\frac{y_S^{(k)} - y_0}{\rho_{0,k}} \quad -\frac{z_S^{(k)} - z_0}{\rho_{0,k}} \quad c \right]$$

a unit vector pointing toward the satellite, with c appended.

4.3 Least-Squares Solution

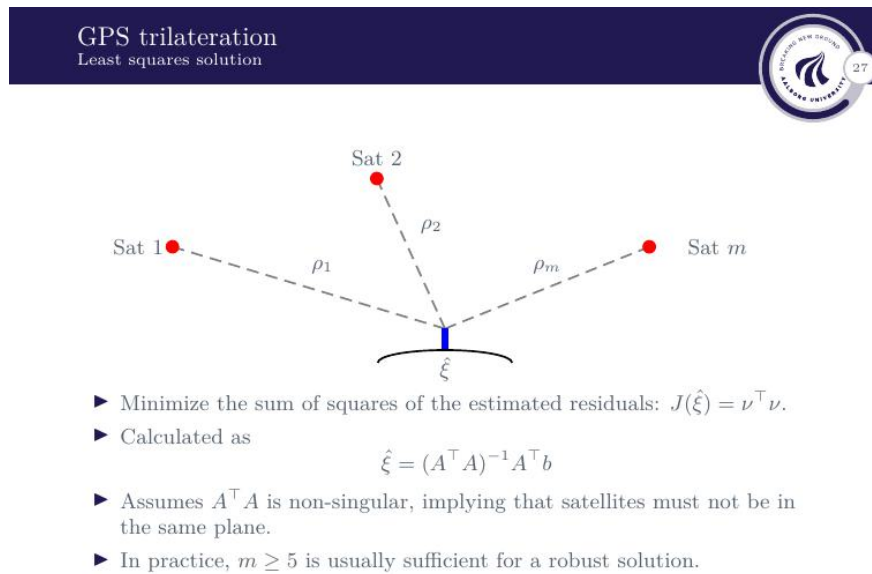


Figure 25: With $m > 4$ satellites the system is overdetermined. Least squares finds $\hat{\xi}$ that minimises the total squared residual across all pseudorange measurements.

With $m > 4$ satellites the system $b = A\hat{\xi} + \nu$ is **overdetermined** — more equations than unknowns. There is no exact solution; instead minimise the sum of squared residuals:

$$J(\hat{\xi}) = \nu^\top \nu = (b - A\hat{\xi})^\top (b - A\hat{\xi})$$

Setting $\partial J / \partial \hat{\xi} = 0$ gives the **normal equation** (same structure as the disc-fitting solution in Lecture 7):

$$\hat{\xi} = (A^\top A)^{-1} A^\top b$$

Why more satellites help, and why geometry matters

With exactly 4 satellites you get an exact solution — but any noise on a pseudorange goes straight into your position error with nothing to average it out. With 5 or more satellites the extra measurements are redundant but *useful*: least squares finds the single (x, y, z, τ) that best fits all measurements simultaneously, averaging out random noise.

Why $m \geq 5$ in practice: $A^\top A$ must be invertible, meaning the 4 columns of A must be linearly independent. This fails if all satellites happen to be coplanar with the receiver — their direction-cosine rows would lie in the same 3D subspace. With 5 or more well-spread satellites this is essentially impossible.

DOP revisited: the conditioning of $A^\top A$ is exactly what DOP quantifies (Section 3.3). Good satellite spread $\rightarrow A^\top A$ well-conditioned $\rightarrow (A^\top A)^{-1}$ small $\rightarrow$ mea-

surement noise produces small position errors (low DOP). Poor spread $\rightarrow A^\top A$ nearly singular $\rightarrow$ same noise causes large errors (high DOP).

4.4 Error Propagation and Covariance

Error computation and Covariance estimates



- Mapping observation 'noise' ν to estimate errors ν_ξ :

$$\nu_\xi = (A^\top A)^{-1} A^\top \nu$$

- Covariance matrix Σ_ξ describes the precision of the estimates:

$$\Sigma_\xi = (A^\top A)^{-1} A^\top \Sigma_\nu A (A^\top A)^{-1}$$

- In particular, if $\Sigma_\nu = \sigma_\nu^2 I$:

$$\Sigma_\xi = \sigma_\nu^2 (A^\top A)^{-1}$$

where σ_ν^2 is the estimated (expected) noise variance for all the observations.

- Diagonal elements of Σ_ξ provide the variance for each coordinate and clock bias.

Figure 26: Propagating pseudorange measurement noise Σ_ν through the least-squares formula gives the position covariance Σ_ξ .

The estimation error (how far $\hat{\xi}$ is from the true solution) propagates from measurement noise ν as:

$$\nu_\xi = \hat{\xi} - \xi_{\text{true}} = (A^\top A)^{-1} A^\top \nu$$

Taking the expectation of the outer product gives the **covariance matrix of position errors**:

$$\Sigma_\xi = (A^\top A)^{-1} A^\top \Sigma_\nu A (A^\top A)^{-1}$$

The equal-noise special case — what the diagonal gives you

The assumption $\Sigma_\nu = \sigma_\nu^2 I$ is a statement about the m **pseudorange measurements** (not the 4 unknowns). It says two things simultaneously:

- *Same noise on every satellite*: every pseudorange has the same variance σ_ν^2 , regardless of which satellite it comes from (elevation angle, signal strength, atmospheric path length, etc.).
- *Independent between satellites*: the noise on one satellite's measurement is unrelated to the noise on another's — the off-diagonals of Σ_ν are zero.

In reality neither is perfectly true — a satellite near the horizon has its signal travel through much more atmosphere and is typically noisier. The general formula $\Sigma_\xi =$

$(A^\top A)^{-1} A^\top \Sigma_\nu A (A^\top A)^{-1}$ handles unequal noise by weighting satellites differently. The equal-noise case is a simplification that gives a clean result:

$$\Sigma_\xi = \sigma_\nu^2 (A^\top A)^{-1}$$

This means the *shape* and *size* of the uncertainty ellipsoid is entirely determined by the satellite geometry $(A^\top A)^{-1}$; the noise level σ_ν^2 just scales the whole thing up or down uniformly. Poor geometry amplifies noise; good geometry suppresses it.

The **diagonal elements** of Σ_ξ are the individual variances (squared 1-sigma errors):

- $[\Sigma_\xi]_{11} = \sigma_x^2$ — variance of x -coordinate error
- $[\Sigma_\xi]_{22} = \sigma_y^2$ — variance of y -coordinate error
- $[\Sigma_\xi]_{33} = \sigma_z^2$ — variance of z -coordinate error
- $[\Sigma_\xi]_{44} = \sigma_\tau^2$ — variance of receiver clock-error estimate

The **off-diagonal elements** describe cross-correlations: e.g. whether an overestimate of x tends to come with an overestimate of y .

4.5 Uncertainty Ellipsoid in Local NEU Coordinates

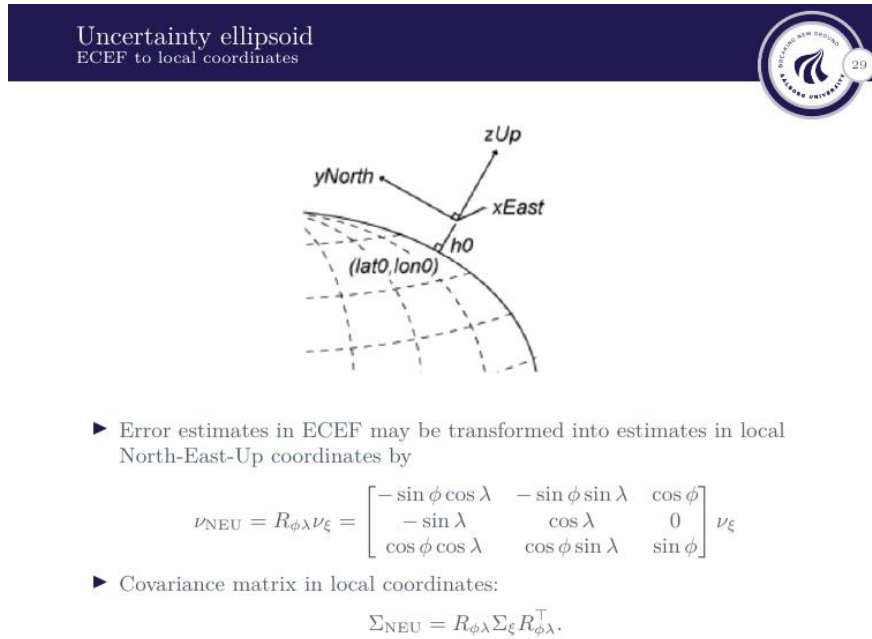


Figure 27: The ECEF covariance Σ_ξ is rotated into the local North-East-Up frame at the receiver's latitude ϕ and longitude λ using $R_{\phi\lambda}$.

The covariance Σ_ξ describes errors in ECEF (x, y, z) coordinates. An error of 5 m in the ECEF x -direction is hard to interpret — it could mean northward, eastward, or upward depending on where on Earth you are. The solution is to rotate the error ellipsoid into the local **North-East-Up** frame at the receiver's position (introduced in Section 2.3). This rotation depends only on **where on Earth the receiver is** — specifically its latitude

ϕ and longitude λ (the same ϕ, λ from Section 2.2). Once you have a GPS fix you know your approximate ϕ and λ , and you use those to build the rotation matrix.

The rotation matrix from ECEF to NEU at latitude ϕ , longitude λ is:

$$R_{\phi\lambda} = \begin{bmatrix} -\sin \phi \cos \lambda & -\sin \phi \sin \lambda & \cos \phi \\ -\sin \lambda & \cos \lambda & 0 \\ \cos \phi \cos \lambda & \cos \phi \sin \lambda & \sin \phi \end{bmatrix}$$

The position error and its covariance in local NEU coordinates are:

$$\nu_{\text{NEU}} = R_{\phi\lambda} \nu_{\xi}, \quad \boxed{\Sigma_{\text{NEU}} = R_{\phi\lambda} \Sigma_{\xi} R_{\phi\lambda}^{\top}}$$

Reading the NEU covariance

The three rows of $R_{\phi\lambda}$ are the unit vectors pointing North, East, and Up from the observer's location:

- **Row 1 (North):** $[-\sin \phi \cos \lambda, -\sin \phi \sin \lambda, \cos \phi]$ — points toward the geographic north pole.
- **Row 2 (East):** $[-\sin \lambda, \cos \lambda, 0]$ — lies in the equatorial plane, pointing east.
- **Row 3 (Up):** $[\cos \phi \cos \lambda, \cos \phi \sin \lambda, \sin \phi]$ — points radially outward from Earth's centre.

After the rotation, the diagonal of Σ_{NEU} is immediately readable:

- $[\Sigma_{\text{NEU}}]_{11} = \sigma_N^2$ — north variance
- $[\Sigma_{\text{NEU}}]_{22} = \sigma_E^2$ — east variance
- $[\Sigma_{\text{NEU}}]_{33} = \sigma_U^2$ — vertical (up) variance

Horizontal uncertainty: $\sigma_H = \sqrt{\sigma_N^2 + \sigma_E^2}$.

Why vertical error is larger: all GPS satellites are above the horizon — there is no satellite directly below the receiver. The Up direction is poorly constrained compared to North and East, so σ_U is typically 2–3 $\times$ larger than σ_H . This is a fundamental geometric limitation of GPS, not a hardware imperfection.

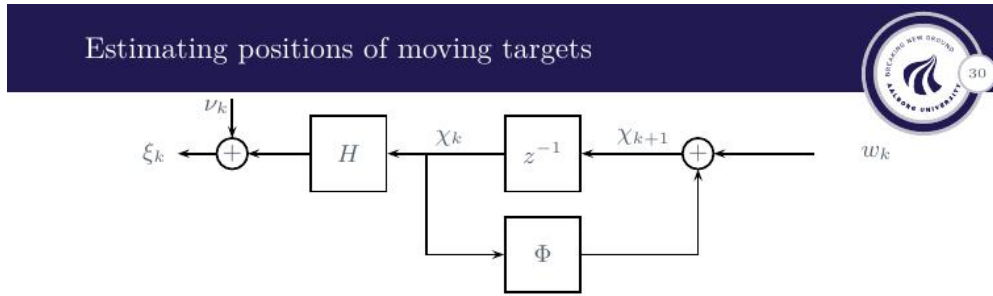
Section 4 — Key Takeaways

- The pseudorange model: $P_S = \rho(t, t_S) + c\tau - c\tau_S$. Correcting the known $c\tau_S$ leaves 4 unknowns: (x, y, z, τ) .
- The nonlinear equations are **linearised** around a guess, giving $b = A\hat{\xi} + \nu$. Each row of A is a unit vector toward one satellite (direction cosines), with c appended for the clock column.
- **Least squares:** $\hat{\xi} = (A^{\top}A)^{-1}A^{\top}b$. Requires $A^{\top}A$ non-singular (satellites not coplanar); $m \geq 5$ gives a robust overdetermined solution.
- **DOP** is the conditioning of $A^{\top}A$: poor geometry $\rightarrow$ large $(A^{\top}A)^{-1} \rightarrow$ measurement

noise amplified into large position errors.

- **Covariance:** $\Sigma_\xi = \sigma_\nu^2(A^\top A)^{-1}$. Diagonal = individual variances for x, y, z, τ .
- Rotating to local NEU via $\Sigma_{\text{NEU}} = R_{\phi\lambda}\Sigma_\xi R_{\phi\lambda}^\top$ gives interpretable uncertainties. Vertical error is $\approx 2\text{--}3\times$ horizontal due to no satellites below the horizon.

Section 5 Tracking Moving Targets



- Obviously, a Kalman estimator can now be used to estimate position and velocity of moving objects based on consecutive GPS observations.
- As usual, if no other information is available, the model above can be used with

$$\chi_k = \begin{bmatrix} x_k \\ y_k \\ z_k \\ v_{x,k} \\ v_{y,k} \\ v_{z,k} \end{bmatrix}, \quad \Phi = \begin{bmatrix} 1 & 0 & 0 & T_s & 0 & 0 \\ 0 & 1 & 0 & 0 & T_s & 0 \\ 0 & 0 & 1 & 0 & 0 & T_s \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad H = \begin{bmatrix} I & 0 \end{bmatrix} \in \mathbb{R}^{3 \times 6}$$

and w, ν the usual Gaussian noise signals.

- In particular, the covariance of ν can be estimated as on the previous slides.

Figure 28: Block diagram of the Kalman filter for GPS-based tracking. GPS pseudoranges provide noisy position observations ξ_k (via the least-squares solution from Section 4). The Kalman filter combines these with a constant-velocity motion model to estimate both position *and* velocity.

So far GPS has been treated as a static problem: estimate the receiver's position at one instant from one batch of pseudoranges. In practice the receiver is often *moving* — a phone, a car, a drone — and we want to track its position and velocity continuously over time.

The solution is a **Kalman filter** applied to consecutive GPS observations. This connects directly to the state-space estimation framework from the Kalman filter lectures: GPS provides the observations, and a motion model provides the prediction between observations.

5.1 State-Space Model

The state vector stacks 3D position and 3D velocity into six components:

$$\chi_k = \begin{bmatrix} x_k \\ y_k \\ z_k \\ v_{x,k} \\ v_{y,k} \\ v_{z,k} \end{bmatrix}$$

State transition matrix Φ (how the state evolves from step k to $k + 1$, assuming constant velocity and sample period T_s):

$$\Phi = \begin{bmatrix} 1 & 0 & 0 & T_s & 0 & 0 \\ 0 & 1 & 0 & 0 & T_s & 0 \\ 0 & 0 & 1 & 0 & 0 & T_s \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Observation matrix H (GPS measures position, not velocity):

$$H = \begin{bmatrix} I & 0 \end{bmatrix} \in \mathbb{R}^{3 \times 6}$$

What Φ and H say about the model

Φ encodes a **constant-velocity motion model**. Read it row by row:

- *Top-left 3×3 identity:* position at the next step starts from current position.
- *Top-right 3×3 block with T_s on the diagonal:* position is updated by adding velocity $\times$ time step ($x_{k+1} = x_k + T_s v_{x,k}$, and similarly for y and z).
- *Bottom-left 3×3 zeros:* velocity does not depend on current position.
- *Bottom-right 3×3 identity:* velocity is assumed constant between steps ($v_{x,k+1} = v_{x,k}$).

Real acceleration (turning, braking) is not modelled explicitly — it is absorbed into the **process noise** w_k . If the target accelerates more than w_k accounts for, the filter lags behind until the GPS observations pull the estimate back.

$H = \begin{bmatrix} I & 0 \end{bmatrix}$ means the GPS measurement gives you the first three state components (position x, y, z) but tells you nothing directly about velocity. The Kalman filter infers velocity from the *change* in position over consecutive GPS fixes.

5.2 Applying the Kalman Filter

The complete discrete-time model is:

$$\chi_{k+1} = \Phi \chi_k + w_k \quad (\text{process / motion model})$$

$$\xi_k = H \chi_k + \nu_k \quad (\text{observation model})$$

where w_k and ν_k are zero-mean Gaussian noise:

- $w_k \sim \mathcal{N}(0, Q)$: process noise covariance Q captures unmodelled accelerations.
- $\nu_k \sim \mathcal{N}(0, R)$: observation noise covariance R is the GPS position error covariance.

Where does the observation noise covariance R come from?

This is exactly Σ_{NEU} (or Σ_ξ restricted to the position components) from Section 4. The full GPS covariance analysis — pseudorange noise propagated through the least-squares formula and rotated into local NEU coordinates — directly gives you R for the

Kalman filter. The two sections are connected: Section 4 produces the measurement uncertainty; Section 5 plugs it into the filter.

In practice R changes from epoch to epoch as the satellite geometry changes (different satellites come into and out of view), so a GPS/Kalman system recomputes R at each time step.

The standard Kalman predict-update cycle then runs at each GPS epoch:

1. **Predict:** propagate the state estimate and its covariance forward using Φ .
2. **Update:** take the new GPS pseudoranges, compute the least-squares position fix $\hat{\xi}_k$ (Section 4), and correct the predicted state toward the observation.
3. The Kalman gain balances trust in the motion model vs. trust in the GPS measurement, weighted by their respective covariances Q and R .

The output at each step is an estimate of *both* position and velocity, smoother than raw GPS alone because the filter integrates information across multiple time steps.

Section 5 — Key Takeaways

- A **Kalman filter** wraps around the GPS least-squares solver to track moving receivers continuously, estimating both position and velocity.
- The **state vector** is $\chi_k = [x, y, z, v_x, v_y, v_z]^\top$; the **constant-velocity model** gives Φ (position updated by $v \cdot T_s$, velocity held constant).
- GPS observes only position: $H = [I \ 0]$. Velocity is inferred from position changes over time.
- The GPS observation noise covariance R comes directly from Section 4's covariance analysis (Σ_ξ or Σ_{NEU}) and is recomputed each epoch as satellite geometry changes.
- Unmodelled accelerations are absorbed by the process noise Q .

Lecture Summary

Summary



- ▶ Trilateration-based sensors use range information to estimate position
- ▶ The Global Positioning System (GPS) is a satellite-based system that has been in operation since the 1980s.
- ▶ It relies on transmission of certain key variables as well as travel time of the data packets themselves to provide range information.
- ▶ Position estimation is performed using local linearization and least squares methods.
- ▶ Errors estimated in global coordinates may be transformed to local coordinates, where they arguably make more sense.

Figure 29: Lecture 8 summary slide.

Lecture 8 — Complete Summary

- **Trilateration-based sensors** estimate position from range measurements to known anchors. Range is obtained via time-of-flight (chirp, acoustic) or signal strength (BLE RSSI), depending on hardware constraints.
- **GPS** is a satellite-based trilateration system operational since the 1980s. Each satellite broadcasts a timing signal and navigation data; the receiver uses the signal travel time to compute pseudoranges.
- **Key variables transmitted by each satellite:** satellite ECEF position (from ephemeris), satellite clock correction τ_S , and the C/A ranging code. Together these allow the receiver to compute the corrected pseudorange $\tilde{P}_k = \rho_k + c\tau$.
- **Position estimation** uses local linearisation (Jacobian) and least squares: $\hat{\xi} = (A^\top A)^{-1} A^\top b$. Four unknowns (x, y, z, τ) require at least 4 satellites; $m \geq 5$ gives an overdetermined, noise-averaging solution.
- **Error analysis:** $\Sigma_\xi = \sigma_\nu^2 (A^\top A)^{-1}$ gives position uncertainty in ECEF; rotating to local NEU via $\Sigma_{\text{NEU}} = R_{\phi\lambda} \Sigma_\xi R_{\phi\lambda}^\top$ gives interpretable horizontal and vertical errors. Vertical error is $\approx 2\text{--}3\times$ horizontal.
- **Moving targets:** a Kalman filter wraps around the GPS solver, using the GPS covariance as observation noise R and a constant-velocity model as the prediction step.